



“ESP32 MOTORES DC”

Mini Proyecto

ING. EN TECNOLOGÍAS DE LA INFORMACIÓN Y COMUNICACIONES

AGOSTO – DICIEMBRE DE 2024

**Martin Arreguin Molina, Juan Carlos Aguilar
Gordillo, Erick Joan Márquez Chávez, Rubén
Andrés Gómez Molina, Brayan Moisés
Cervantes Barrera**

TI21110075

TI21110048

TI21110054

CTI2211815

TI21110118

17/10/2024



**INGENIERÍA
EN TECNOLOGÍAS
DE LA INFORMACIÓN
Y COMUNICACIONES**

TABLA DE CONTENIDO

TABLA DE CONTENIDO	1
AGRADECIMIENTOS	2
RESUMEN	3
CAPITULO 1. PROTOCOLO DEL PROYECTO	4
1.1 Introducción	4
1.2 Planeamiento del problema	5
1.3 Objetivo general	5
1.4 Objetivos específicos	5
1.5 Justificación	7
1.6 Alcance del proyecto	8
1.7 Limitaciones	8
1.8 Cronograma preliminar de actividades	8
1.9 Descripción Detallada de Actividades	9
1.10 Lugar donde se realizará el proyecto	10
1.11 Conclusiones	10
CAPITULO 2. MARCO TEORICO	11
2.1 Motor DC	11
2.2 Modulo Puente H	13
2.3 Esp32 s3 wroom 1	14
2.4 Wokwi	19
2.5 VS Code con PlatformIO y Arduino Framework	22
2.6 Drivers de Espressif	24
2.7 Dip Switch	26
2.8 Resistencias	27
2.9 Potenciómetro de 100k	29
CAPITULO 3. Desarrollo	31
3.1 CONFIGURACION DEL ENTORNO	31
3.2 Circuito de conexiones de la parte de lógica y control (ESP32)	36
3.3 Montaje del sistema de potencia con Puente H	39
3.4 Compilación y programación del sistema, ya diagramado	41
CAPITULO 4. COMPETENCIAS DESARROLLADAS	51
CAPITULO 5. Resultados y conclusiones	52
5.1 RESULTADOS	52
5.2 CONCLUSIONES DEL PROYECTO	53

Bibliografías.....54

TABLA DE ILUSTRACIONES

Figura 1 Automatización de sistemas	4
Figura 2 Diagrama ESP32	6
Figura 3 Ing en TIC'S	7
Figura 4 Motor DC	11
Figura 5 Modulo puente H	13
Figura 6 Diagrama modulo puente H	14
Figura 7 Diagrama ESP32 s3 wroom 1	15
Figura 8 Procesador ESP32	16
Figura 9 Logotipo Oficial de wokwi	19
Figura 10 Simulador en linea Wokwi	20
Figura 11 Pantalla de VS Code con PlatformIO para el ESP32	23
Figura 12 ESP 32 componentes.....	25
Figura 13 Dip Switch en circuito	27
Figura 14 Resistencias en un circuito	28
Figura 15 Potenciómetro estructura	30
Figura 16 DESCARGA DE EXTENSION (platformio.org, 2024).....	31
Figura 17 ITESS Equipo Menú de inicio de PlatformIO	31
Figura 18 Esspresif 32 Instalación de plataformas	32
Figura 19 Plataformas instaladas dentro de PlataformIO	32
Figura 20 Proyectos abiertos.....	33
Figura 21 Menú de proyectos, aquí se compilan los proyectos y se suben a las placas	33
Figura 22 Edicion del circuito en WOKWI	34
Figura 23 Captura de pantalla del proyecto en VS code offline.....	34
Figura 24 toml para simulación.....	35
Figura 25 Plataformio.ini.....	35
Figura 26 colocación en protoboard (jBo)	36
Figura 27 Fragmento que representa configuración de pines.....	36
Figura 28 esquema de conexión del puente H	37
Figura 29 Conexiones de pines que son correspondientes al motor	37
Figura 31 conf. de resistencias salida al modulo.....	37
Figura 32 pull down activo señal alta	38
Figura 33 pull down inactivo señal baja	38
Figura 34 configuración del switch en pull down	38
Figura 35 Método ConfigurarPines() , entradas tipo INPUT_PULLDOWN	39
Figura 36 Conexión (Medium)GND común ejemplo.....	39
Figura 37 Progreso de circuito.....	40
Figura 38 Progreso de circuito.....	40
Figura 39 motor soldadura de pines.....	40
Figura 40 soldadura de pines y protección con cinta de aislar	40
Figura 41 Administrador de dispositivos (ESPRESSIF).....	41
Figura 42 Conexión del computador (hutscape.com).....	41
Figura 43 USB serial.....	41
Figura 44 mensaje de compilación exitosa , propietario	42
Figura 45 MENSAJE de programación de placa exitoso (Bassi, 2021).....	42
Figura 46 Evidencias del proyecto en funcionamiento.....	52

AGRADECIMIENTOS

Agradecemos a los profesores que nos impartieron las distintas clases, en las cuales fuimos adquiriendo conocimientos recreativos y claros para poder hacer las cosas, tanto el proyecto que fuimos elaborando y la manera de como documentar las cosas para un fin creativo y personalizable exitoso

RESUMEN

En este proyecto se desarrolla un sistema de control , en el lenguaje c++, con el framework de Arduino y la plataforma de Visual Studio Code , se implementan conceptos de programación orientada a objetos, encapsulación, el uso de arreglos y estructuras para facilitar la modificación del proyecto, el uso de programación orientada a métodos, el uso de entornos gráficos para diseñar el circuito para el prototipo, la planificación y presupuestación de un proyecto , uso de sistemas embebidos de baja potencia para controlar elementos de potencia. En este documento se aplican conocimientos de electrónica y se da el código fuente de control del proyecto, se da la plataforma en la que se desarrolló , la metodología y el resultado.

CAPITULO 1. PROTOCOLO DEL PROYECTO

1.1 Introducción

La automatización consiste en usar la tecnología para realizar tareas con muy poca intervención humana. Se puede implementar en cualquier sector en el que se lleven a cabo tareas repetitivas. Sin embargo, es más común en aquellos relacionados con la fabricación, la robótica y los automóviles, así como en el mundo de la tecnología: en el software para la toma de decisiones empresariales y los sistemas de TI. (RedHat, 2024) En el ámbito de la automatización de sistemas, la implementación de mecanismos que optimicen procesos mediante el uso de motores eléctricos y controles básicos es esencial para diversas aplicaciones industriales y educativas. Este proyecto se centra en el diseño y desarrollo de un sistema que permite controlar dos motores de corriente continua (DC) utilizando un potenciómetro para variar su velocidad, así como interruptores para gestionar la dirección de giro y detener por completo los motores. El enfoque principal del proyecto es proporcionar un sistema sencillo y funcional que pueda servir tanto como herramienta de aprendizaje en el área de automatización como una base para desarrollar soluciones más complejas.



*Figura 1 Automatización de sistemas
(teknei, 2024)*

La automatización en informática se refiere al uso de sistemas y software para realizar tareas automáticamente, sin necesidad de intervención humana continua

1.2 Planeamiento del problema

El control de motores DC con precisión es una necesidad en múltiples aplicaciones industriales y educativas. Sin embargo, muchas soluciones existentes son costosas o complejas para quienes están comenzando en el área de automatización. Este proyecto aborda esa problemática mediante el desarrollo de un sistema económico y funcional basado en el ESP32, que puede ser fácilmente replicado y adaptado.

1.3 Objetivo general

Desarrollar un sistema de control de motores DC utilizando un ESP32, que permita regular la velocidad, cambiar la dirección de giro y detener los motores de forma sencilla y eficiente.

1.4 Objetivos específicos

Diseñar e implementar un sistema de control basado en ESP32:

- Crear un esquema eléctrico que integre el ESP32, el potenciómetro, los switches, y los motores DC.
- Asegurar la compatibilidad de voltajes y corrientes entre el ESP32 y los motores mediante el uso de transistores o drivers adecuados (como un puente H).
- Garantizar la correcta alimentación del sistema para evitar fluctuaciones que afecten el funcionamiento.

Programar el ESP32 para el control de velocidad:

- Configurar el ESP32 para leer señales analógicas del potenciómetro a través de sus pines ADC (convertidor analógico-digital).
- Mapear la señal del potenciómetro a un rango PWM (Modulación por Ancho de Pulso) adecuado para el control de velocidad de los motores.
- Implementar un código optimizado que permita un ajuste suave y preciso de la velocidad.

implementar la funcionalidad de cambio de dirección:

- Configurar entradas digitales en el ESP32 para detectar el estado del switch de dirección.
- Diseñar un algoritmo que invierta la polaridad del voltaje aplicado a los motores mediante un puente H o relés, según el estado del switch.
- Verificar la transición fluida entre los modos de avance y retroceso, evitando cortes bruscos que puedan dañar el motor.

Incorporar un sistema de paro total de los motores:

- Configurar una entrada digital en el ESP32 para detectar el estado del switch de paro total.
- Programar el ESP32 para desactivar las señales PWM y detener por completo los motores al activarse el switch.

- Asegurar que el sistema pueda reanudarse sin problemas tras el paro.

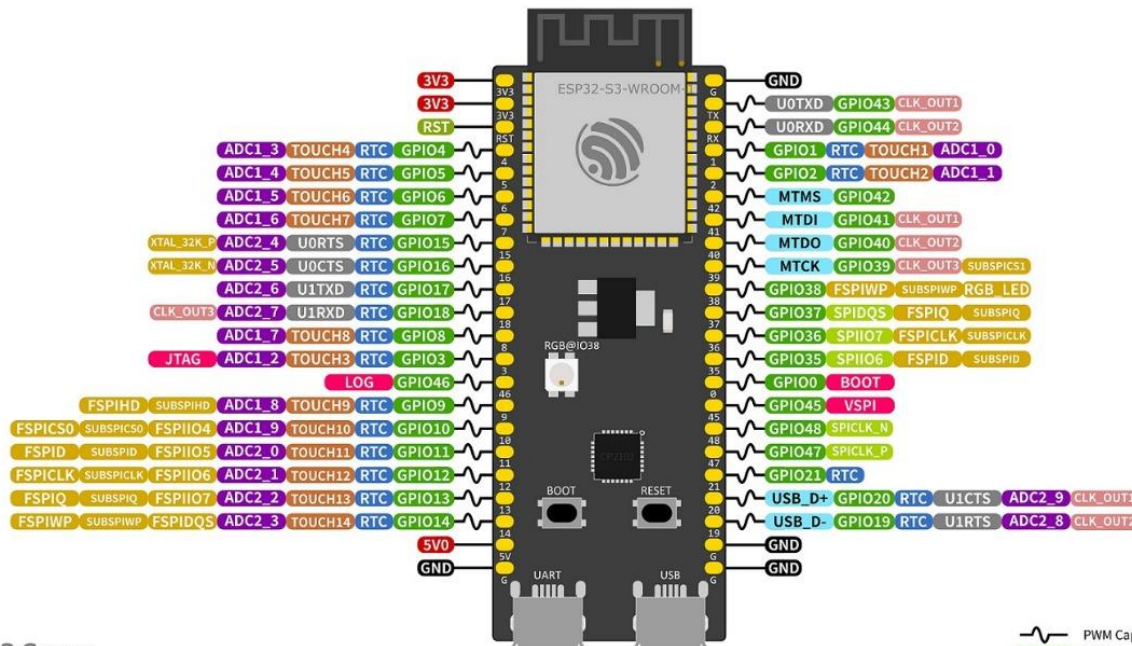
Pruebas y optimización del sistema:

- Realizar pruebas individuales para cada componente del sistema: lectura del potenciómetro, cambio de dirección y paro total.
- Evaluar el rendimiento del sistema en diferentes escenarios, como velocidades altas y bajas, cambios rápidos de dirección, y activación repetida del paro total.
- Ajustar parámetros del código o del circuito para mejorar la estabilidad y precisión del control.

Documentar el proyecto para replicación futura:

- Crear diagramas esquemáticos del circuito y explicar la funcionalidad de cada componente.
- Proporcionar el código completo y comentado del ESP32, incluyendo instrucciones para cargarlo al microcontrolador.
- Redactar un informe detallado con los pasos de construcción, los resultados de las pruebas, y sugerencias para mejoras o expansiones del sistema.

ESP32-S3-DevKitC-1



ESP32-S3 Specs

32-bit Xtensa® dual-core @240MHz
 Wi-Fi IEEE 802.11 b/g/n 2.4GHz + BLE 5 Mesh
 512 KB SRAM (16 KB SRAM in RTC)
 384 KB ROM
 45 GPIOs, 4x SPI, 3x UART, 2x I2C,
 14x Touch, 2x I2S, RMT, LED PWM, USB-OTG,
 TWAI®, 2x 12-bit ADC, 1x LCD interface, DVP

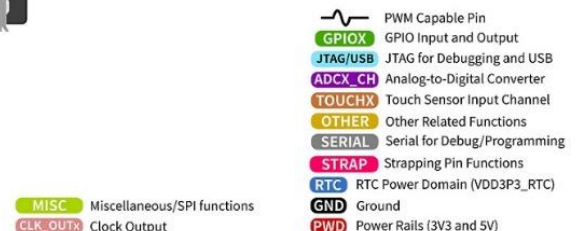


Figura 2 Diagrama ESP32

1.5 Justificación

El uso de sistemas de control basados en microcontroladores, como el ESP32, constituye una solución accesible y eficiente para diversas aplicaciones en el ámbito de la automatización. Este proyecto se justifica principalmente por su relevancia educativa y tecnológica, ya que fomenta el aprendizaje práctico de habilidades esenciales para la programación y manejo de microcontroladores, así como el control de motores y la gestión de señales analógicas y digitales. Estos conceptos son fundamentales en la formación de ingenieros en áreas como la robótica, mecatrónica y electrónica. Se hace referencia en la figura 3



Figura 3 Ing en TIC'S
(Toluca, Ingenieria TIC'S, 2024)

Además, el proyecto destaca la versatilidad del ESP32, un microcontrolador que combina capacidad de procesamiento, conectividad Wi-Fi/Bluetooth y múltiples entradas y salidas. Esta capacidad permite manejar funciones como el control de velocidad, cambio de dirección y paro total de motores, lo que demuestra su potencial como herramienta poderosa para proyectos tanto académicos como industriales.

El impacto del proyecto va más allá del ámbito educativo, ya que puede servir como base para aplicaciones más avanzadas. Por ejemplo, podría integrarse con sensores para sistemas de monitoreo o con conectividad inalámbrica para control remoto. Incluso en entornos industriales, este sistema puede adaptarse para procesos automatizados de mayor escala, lo que subraya su escalabilidad y utilidad práctica.

Por otra parte, el proyecto contribuye al desarrollo de soluciones de bajo costo, ofreciendo una alternativa económica y personalizable para problemas comunes en la automatización. Esto fomenta la innovación accesible para pequeñas empresas, instituciones educativas y entusiastas de la tecnología. También tiene un impacto ambiental positivo, al promover

sistemas que optimizan el consumo energético mediante el uso eficiente de componentes electrónicos.

1.6 Alcance del proyecto

El proyecto tiene como objetivo principal implementar un sistema funcional basado en el microcontrolador ESP32 para controlar dos motores de corriente continua. Este sistema permite regular la velocidad mediante un potenciómetro, cambiar la dirección de giro con un switch y detener los motores completamente con otro switch de paro total. Además de demostrar la versatilidad del ESP32, este trabajo busca integrar teoría y práctica para cumplir con los requisitos de la materia *Automatización de Sistemas*. Su desarrollo incluye el diseño del circuito, la programación y pruebas funcionales, garantizando una solución eficiente y bien documentada. Aunque el enfoque actual está en cumplir con fines educativos, el sistema puede adaptarse en el futuro para aplicaciones más avanzadas, como control remoto o integración con sensores. De esta forma, el proyecto no solo contribuye al aprendizaje, sino también a la aplicación práctica de tecnologías accesibles en automatización.

1.7 Limitaciones

- Falta de tiempo
- Acceso limitado a cursos especializados
- Motivación o constancia insuficiente (Indisciplina)
- Dificultad en encontrar ambientes de práctica real
- Falta de retroalimentación continua (dudas)

1.8 Cronograma preliminar de actividades

Actividades por desarrollar	Semana 1	Semana 2	Semana 3	Semana 4	Semana 5	Semana 6
Investigación y selección de componentes	2 horas					
Diseño del circuito en software (Fritzing o similar)	3 horas					
Adquisición de materiales	1 hora					
Montaje del circuito en protoboard		3 horas				
Programación inicial del ESP32		4 horas				
Pruebas de control de velocidad con potenciómetro			2 horas			
Configuración del cambio de dirección con switch			2 horas			

Implementación del switch de paro total			2 horas			
Ajustes y optimización del código			1 hora	2 horas		
Pruebas integrales del sistema				3 horas		
Documentación de los resultados					3 horas	
Preparación de la presentación del proyecto					2 horas	
Ensayo de la presentación y correcciones finales						2 horas

1.9 Descripción Detallada de Actividades

1.-Investigación y selección de componentes:

En esta etapa inicial, se identifican los componentes necesarios para el proyecto, como el ESP32, motores de corriente continua, potenciómetro, switches y otros elementos electrónicos (resistencias, cables, protoboard). Además, se analizan las especificaciones técnicas de cada componente para garantizar su compatibilidad. También se investiga sobre el control de motores DC y la programación del ESP32 para comprender los principios que guiarán el diseño.

2. Diseño del circuito eléctrico:

Se realiza un diseño preliminar del circuito utilizando herramientas de software como Fritzing o Tinkercad. Este diseño incluye la conexión del ESP32 con los motores, el potenciómetro y los switches, considerando el uso de transistores o drivers como el L298N para manejar la corriente de los motores. Este paso asegura que el circuito sea funcional antes de proceder al montaje físico.

3. Adquisición de materiales y preparación:

Una vez definido el diseño, se adquieren los componentes en tiendas especializadas o en línea. Se verifica que los materiales lleguen en buen estado y se organizan para el montaje. En paralelo, se preparan los instrumentos necesarios, como multímetros, cautines y herramientas para trabajar el protoboard.

4. Montaje del circuito en protoboard:

En esta etapa, el circuito se ensambla físicamente en una protoboard, siguiendo el diseño previamente realizado. Se conectan los motores, el potenciómetro y los switches al ESP32, asegurando que cada conexión sea estable y cumpla con el esquema eléctrico. Se realizan pruebas de continuidad para garantizar que no haya errores en las conexiones.

5. Programación del ESP32:

Se desarrolla el código que controlará los motores. Este incluye módulos para la generación de señales PWM, la lectura del potenciómetro y los switches, y las funciones de control de

dirección y paro total. El código se escribe en el entorno de desarrollo Arduino IDE o PlatformIO, y se realiza la carga inicial al ESP32 para verificar su funcionamiento básico.

6. Pruebas individuales de cada módulo:

- Control de velocidad: Se evalúa la respuesta del sistema al variar la posición del potenciómetro, verificando que los motores cambien su velocidad de manera proporcional.
- Cambio de dirección: Se prueba el switch de dirección, asegurando que los motores respondan correctamente invirtiendo su giro.
- Paro total: Se activa el switch de paro total, comprobando que los motores se detengan inmediatamente sin fallos.

7. Integración y pruebas finales:

Una vez probados los módulos individualmente, se integran en un sistema completo. Se realizan pruebas funcionales bajo diferentes condiciones, como cambios rápidos de dirección o ajustes de velocidad, para garantizar que el sistema sea estable y confiable.

8. Optimización del sistema:

Con base en las pruebas, se realizan ajustes en el código o en el montaje físico para mejorar la precisión, estabilidad y eficiencia del sistema. Esto incluye ajustes en las señales PWM, la configuración del ESP32 o el uso de componentes adicionales si es necesario.

9. Documentación del proceso:

Se elabora una documentación detallada que incluye el diseño del circuito, el código del programa, las pruebas realizadas y los resultados obtenidos. Este documento también describe los problemas encontrados durante el desarrollo y las soluciones implementadas, proporcionando un manual completo para replicar el proyecto.

10. Preparación y ensayo de la presentación:

Finalmente, se organiza una presentación del proyecto, incluyendo gráficos, diagramas y videos que muestren su funcionamiento. Se realiza un ensayo para garantizar una exposición clara y precisa durante la entrega del trabajo en la materia Automatización de Sistemas.

1.10 Lugar donde se realizará el proyecto

El proyecto se llevará a cabo en las instalaciones del Instituto Tecnológico de Estudios Superiores de Salvatierra (ITESS), ubicado en Carretera Salvatierra-Acámbaro km 1.6, Salvatierra, Guanajuato, México. Este proyecto es parte de las actividades académicas dentro de la asignatura y será desarrollado por el equipo que nosotros hicimos con ayuda del profesor asignado de la materia.

1.11 Conclusiones

Este proyecto demuestra cómo el uso de un ESP32 puede simplificar y optimizar el control de motores DC, ofreciendo una solución accesible y funcional. Además, fomenta el aprendizaje práctico en electrónica y programación, estableciendo una base sólida para proyectos más avanzados en el área de automatización.

CAPITULO 2. MARCO TEORICO

2.1 Motor DC

¿Qué es un motor DC y como funciona?

Lo primero que podemos destacar del motor DC es que también es conocido con el nombre de motor DC, con la particularidad de que tiene la principal función de transformar la energía eléctrica en energía mecánica. Para comprender el funcionamiento de este tipo de motor es importante aclarar que tienen la capacidad de generar un campo magnético que sobre el rotor y, de este modo, será capaz de convertir la energía eléctrica en energía mecánica.

Otro detalle que hay que considerar cuando hablamos de un motor de corriente continua es que encontraremos ejemplos de distintos tipos según como se conecten los bobinados. Además, podemos determinar que un motor DC funcionan a través de corriente continua y, a su vez, son máquinas de conmutación mecánica. (SDI, 2020)



Figura 4 Motor DC
(angelmicelti, 2023)

¿Qué características tiene un motor DC?

Es momento ahora de conocer las principales características y particularidades que encontramos en un motor de corriente continua, puesto que nos servirá para ampliar la información relacionada con este tipo de motor.

Bien, uno de los aspectos más representativos es que existen de todo tipo de tamaños y potencias. De hecho, los podemos encontrar en juguetes o electrodomésticos, así como también existen variedades mucho más grandes y que sirven para impulsar todo tipo de vehículos o ascensores, entre otros ejemplos.

Por otra parte, otra de las principales características que nos ofrece el motor de corriente continua es que cuentan con una gran demanda en el sector industrial dado que es muy sencillo regular la velocidad del motor, puesto que únicamente cambiando la polaridad conseguiremos modificar el sentido de giro.

Para entender la repercusión que está teniendo el motor de corriente continua es indispensable tener en cuenta las numerosas ventajas que nos ofrece, siendo importante aclarar que son muy fáciles de controlar, tanto en acciones tan indispensables como la puesta en marcha o como el frenado.

Conocemos ahora las principales partes que tenemos en un motor DC y destacaremos que existen dos partes principales; estator y rotor.

El estator es la parte fija y es inmóvil, con la particularidad de que suele estar formado por diferentes imanes y bobinas. En cambio, el rotor, es la parte móvil que gira dentro del estator, siendo un elemento que suele estar compuesto por un eje con capas magnéticas o, en su defecto, por bobinas. (SDI, 2020)

¿Qué ventajas tiene un motor DC?

Ya hemos explicado que el motor de corriente continua disfruta de un gran reclamo en la actualidad, algo que no es de extrañar si tenemos en cuenta la amplia variedad de ventajas que nos ofrece esta variedad de motor.

Por este motivo, es momento de mencionar las principales ventajas que nos propone este tipo de motor de corriente continua. ¡No te las pierdas!

Bien, una de las ventajas más interesantes de un motor de corriente continua es que tiene la capacidad de poder a trabajar a velocidades bajas, siendo esta una de las principales razones por las que existen motores DC de todo tipo de tamaños.

Hay que resaltar, también, que la regulación de la velocidad es mucho más simple y económica en comparación con los motores de corriente alterna, de modo que estamos ante otra interesante ventaja que no podemos olvidar a la hora de hablar de los motores de corriente continua.

Si aún no estás convencido de las ventajas que nos ofrecen los motores de corriente continua es necesario aclarar que tendrás la opción de poder usarlos con reductores para numerosas aplicaciones y, de este modo, conseguirán una mayor fuerza de giro.

Finalmente, no podemos finalizar este listado de ventajas sin destacar que los motores DC se pueden alimentar con energía almacenada en baterías o a través de energía fotovoltaica.

En conclusión, son muchas las razones por las cuáles cada vez son más las personas que apuestan este tipo de motores, así como se espera que esta tendencia no haga más que crecer en un futuro cercano, un aspecto que no nos extraña si tenemos en cuenta las numerosas ventajas que nos ofrecen estos motores. (SDI, 2020)

2.2 Modulo Puente H

¿Qué es el Módulo Puente H?

El driver puente H L298N es el modulo más utilizado para manejar motores DC de hasta 2 amperios. El chip L298N internamente posee dos puentes H completos que permiten controlar 2 motores DC o un motor paso a paso bipolar/unipolar el módulo permite controlar el sentido y velocidad de giro de motores mediante señales TTL que se pueden obtener de microcontroladores y tarjetas de desarrollo como Arduino, Raspberry Pi o Launchpads de Texas Instruments. El control del sentido de giro se realiza mediante dos pines para cada motor, la velocidad de giro se puede regular haciendo uso de modulación por ancho de pulso (PWM por sus siglas en inglés) tiene integrado un regulador de voltaje LM7805 de 5V encargado de alimentar la parte lógica del L298N, el uso de este regulador se hace a través de un Jumper y se puede usar para alimentar la etapa de control.



Figura 5 Modulo puente H
(Geekfactory, 2018)

Especificaciones Técnicas

- Chip: L298N
- Canales: 2 (soporta 2 motores DC o 1 motor PAP)
- Voltaje lógico: 5V
- Voltaje de potencia (V motor): 5V - 35V DC
- Consumo de corriente (lógico): 0 a 36mA
- Capacidad de corriente: 2A (picos de hasta 3A)
- Potencia máxima: 25W

- Dimensiones: 43 * 43 * 27 mm
- Peso: 30g

¿Cómo funciona el Módulo Puente H?

El puente H es un circuito electrónico que permite controlar la polaridad del voltaje aplicado a una carga que habitualmente es un motor, bobina o solenoide. Mediante el puente H es posible controlar la dirección de giro del motor hacia adelante o hacia atrás.

Los puentes H se utilizan ampliamente en la robótica y en general en cualquier aplicación donde se necesite controlar la dirección y velocidad de giro de motores de corriente directa.

El diagrama simplificado de un puente H puede apreciarse en la siguiente imagen. Utilizando este modelo simplificado podemos explicar el funcionamiento de un puente H en pocas palabras.

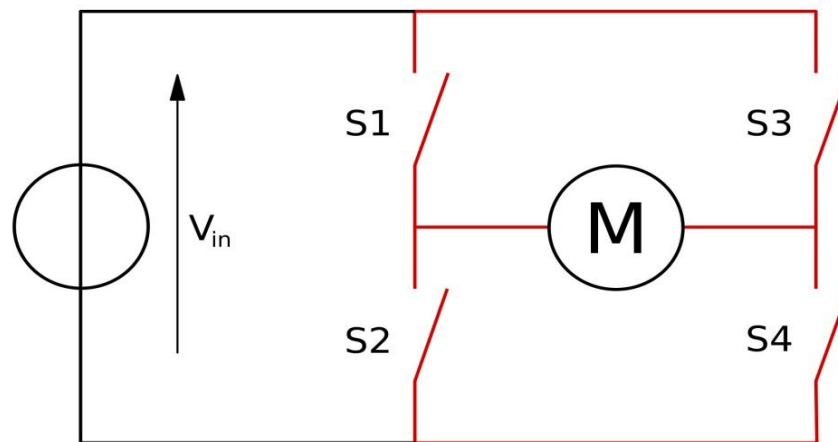


Figura 6 Diagrama modulo puente H
(Geekfactory, 2018)

2.3 Esp32 s3 wroom 1

El ESP32-S3-WROOM-1 es un módulo de la serie ESP32, desarrollado por Espressif Systems, que forma parte de la familia de microcontroladores de bajo consumo y alto rendimiento, muy populares para proyectos de IoT (Internet of Things). Este modelo en particular tiene mejoras en el procesamiento de señales, capacidades de IA (inteligencia artificial), y un mayor rendimiento en comparación con otros microcontroladores de la misma familia.

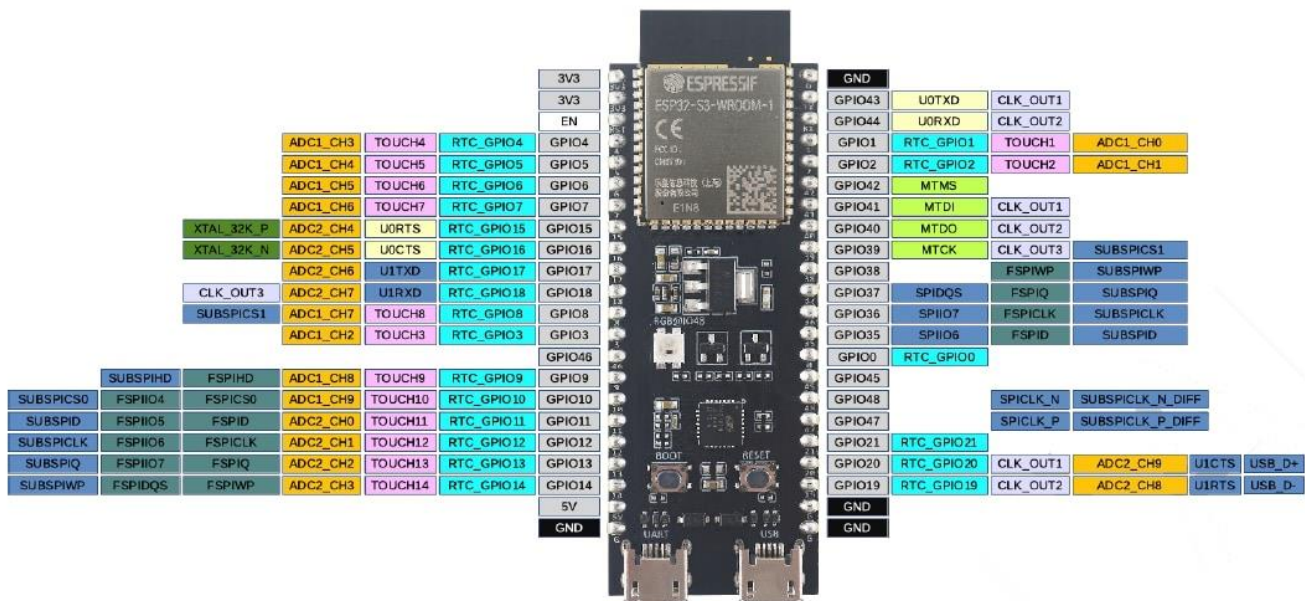




Figura 8 Procesador ESP32
(Mouser, 2021)

Características Principales del ESP32-S3-WROOM-1:

1. Procesador:

- CPU: Dual-core (dos núcleos) con arquitectura Xtensa® 32-bit LX7 de Espressif.
- Velocidad de reloj: Hasta 240 MHz.
- Caché: 512 KB SRAM, que mejora la velocidad de procesamiento.

2. Memoria:

- Flash: 4 MB de memoria flash externa, ampliable según la configuración del módulo.
- RAM: 512 KB de SRAM, que se utiliza para la ejecución de procesos de manera eficiente.

3. Conectividad:

- Wi-Fi: Soporta 802.11 b/g/n (Wi-Fi 4), lo que permite la conectividad a redes de alta velocidad.

- Bluetooth: Soporta Bluetooth 5.0 Low Energy (BLE), útil para conectar dispositivos de bajo consumo y de corto alcance.
- Soporte para múltiples protocolos: Incluye soporte para protocolos como MQTT, HTTP, CoAP y más, ideal para aplicaciones IoT.

4. Procesamiento de señales:

- El ESP32-S3 tiene aceleradores dedicados a procesamiento de señales (como aceleradores de vector y DSP), lo que lo hace ideal para tareas de inteligencia artificial (IA) y procesamiento de voz o imágenes.

5. Puertos y Interfaces:

- GPIOs: 34 pines GPIO (algunos de ellos con funciones específicas como ADC, DAC, PWM, I2C, SPI, UART, etc.).
- ADC/DAC: 12 bits de resolución en las entradas analógicas (ADC), y 8 bits de resolución en las salidas analógicas (DAC).
- I2C, SPI, UART: Soporta múltiples interfaces para la conexión de dispositivos periféricos.
- PWM (Pulse Width Modulation): Utilizado para controlar la velocidad de motores, iluminación LED, entre otros.
- Canal de I2S (Inter-IC Sound): Soporta la transmisión de señales de audio.
- Soporte de pantallas táctiles y de gráficos: A través de interfaces como SPI y I2S.

6. Consumo energético:

- Bajo consumo: El ESP32-S3 es eficiente en términos de consumo de energía, con modos de suspensión que permiten que los dispositivos IoT funcionen durante largos periodos con baterías pequeñas.
- Modos de energía: Modo de suspensión profunda, modo de suspensión activa, y modo de ejecución, lo que optimiza el consumo de energía.

7. Interfaz de programación:

- USB-C: El módulo es compatible con un puerto de programación y depuración a través de USB-C para facilitar la conexión al entorno de desarrollo.
- Programación fácil: Compatible con Arduino IDE, ESP-IDF (framework oficial de Espressif), y otros entornos de desarrollo como PlatformIO.

8. Otros aspectos importantes:

- Soporte de cámara: El ESP32-S3 también tiene soporte para cámaras y aplicaciones de visión computacional mediante interfaces como DVP (Digital Video Port).
- Hardware de seguridad: Incluye soporte para encriptación AES, SHA, RSA, y otros algoritmos de seguridad, lo que lo hace adecuado para aplicaciones que requieren seguridad en la comunicación.

¿Cómo funciona el ESP32-S3-WROOM-1?

1. Inicialización:

- El ESP32-S3-WROOM-1 arranca a partir de la memoria flash donde reside el firmware que define su comportamiento. Cuando se enciende, el microcontrolador comienza a ejecutar el código cargado desde la memoria externa.

2. Conectividad Wi-Fi y Bluetooth:

- En el caso de Wi-Fi, el ESP32-S3 se conecta a una red, lo que permite la comunicación con otros dispositivos a través de internet. Para Bluetooth, el dispositivo puede actuar como maestro o esclavo, permitiendo la interconexión con otros dispositivos BLE como teléfonos o sensores.

3. Uso de GPIO:

- Los pines GPIO del ESP32-S3 pueden configurarse como entradas o salidas, y dependiendo de la programación, se pueden utilizar para interactuar con sensores, botones, pantallas, relés, entre otros periféricos.

4. Control de señales de entrada y salida:

- A través de interfaces como I2C, SPI, UART, y PWM, el ESP32-S3 puede controlar y recibir señales desde otros dispositivos electrónicos. Esto lo hace muy adecuado para tareas de automatización, control de motores, y adquisición de datos.

5. Procesamiento de datos:

- El ESP32-S3 cuenta con aceleradores de procesamiento de señales y capacidades para IA. Esto le permite realizar tareas de procesamiento de imágenes, reconocimiento de voz, y otras aplicaciones que requieren cálculos complejos y rápidos, sin depender de un servidor central.

Puertos y Conexiones del ESP32-S3-WROOM-1:

- GPIO Pins: Hasta 34 pines GPIO que permiten conexiones con sensores y actuadores.
- ADC (Analógico a Digital): Hasta 18 canales de entrada analógica de 12 bits de resolución.
- DAC (Digital a Analógico): 2 canales de salida analógica de 8 bits de resolución.
- I2C: Soporte para comunicación con dispositivos I2C.
- SPI: Soporta conexiones SPI para comunicación de alta velocidad.
- UART: Permite comunicaciones serie, comúnmente utilizada para depuración o conexión con otros dispositivos.
- PWM: Control de señal de ancho de pulso para motores, LEDs, y otros dispositivos.
- I2S: Utilizado para audio o comunicación con dispositivos de alta velocidad.

¿Cómo se usa el ESP32-S3-WROOM-1?

1. Entorno de Desarrollo:
Se puede usar con Arduino IDE, PlatformIO, o el ESP-IDF (el framework oficial de Espressif). Arduino IDE es una opción fácil y popular para principiantes, mientras que ESP-IDF ofrece más control sobre el hardware y permite acceder a características avanzadas.
2. Programación:
El código para controlar el ESP32-S3 se carga en el módulo a través de un puerto USB. El código puede incluir funcionalidades como el control de GPIO, la gestión de Wi-Fi, y la implementación de protocolos de comunicación.
3. Conexión con otros dispositivos:
El ESP32-S3 se puede conectar a otros dispositivos como sensores, cámaras, o pantallas mediante las interfaces mencionadas (SPI, I2C, UART), lo que lo convierte en una excelente opción para proyectos de automatización, monitoreo, control, y dispositivos portátiles.

El ESP32-S3-WROOM-1 es un componente versátil y potente para el desarrollo de aplicaciones IoT, permitiendo comunicaciones inalámbricas, control de dispositivos, y procesamiento local de señales, todo con un bajo consumo energético.

2.4 Wokwi

¿Qué es Wokwi?

Wokwi es un simulador en línea que ofrece un entorno interactivo para diseñar, programar y simular proyectos basados en Arduino. Al centrarse en esta popular plataforma de desarrollo, Wokwi proporciona a los entusiastas de Arduino una herramienta eficiente para experimentar sin la necesidad de hardware físico.



Figura 9 Logotipo Oficial de wokwi
(Luisllamas, 2022)

Wokwi es un simulador de electrónica en línea gratuito que permite simular:

- Arduino
- ESP32
- STM32
- Y otras placas, partes y sensores

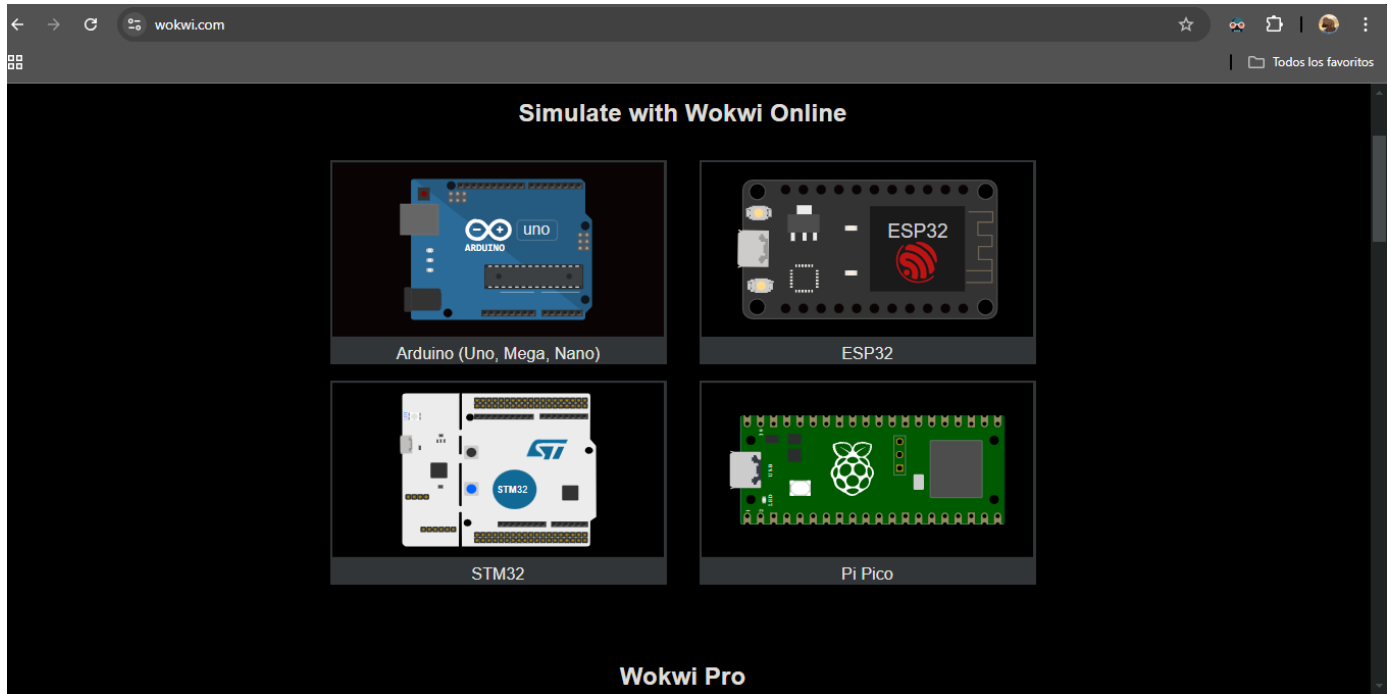


Figura 10 Simulador en línea Wokwi
(wokwi, 2024)

Características de Wokwi:

1. Simulación de Circuitos Electrónicos:

- Wokwi ofrece un entorno de simulación donde puedes construir circuitos electrónicos de forma visual, agregando componentes como resistencias, LEDs, transistores, motores, sensores, pantallas, y más.
- La plataforma simula el comportamiento real de los circuitos, permitiéndote ver cómo funcionan sin necesidad de un montaje físico.

2. Soporte para Microcontroladores Populares:

- Arduino: Wokwi soporta todas las placas populares de Arduino, como Arduino Uno, Mega, Nano, etc.
- ESP32 y ESP8266: Además de las placas de Arduino, Wokwi también es compatible con microcontroladores como el ESP32 y ESP8266, lo que lo convierte en una excelente opción para proyectos de IoT.

- Otros Microcontroladores: También puedes simular placas de otros microcontroladores y dispositivos, como el STM32, Raspberry Pi Pico, y más.

3. Editor de Código en Línea:

- Wokwi permite escribir, editar y cargar código directamente en el navegador. El editor soporta varios lenguajes de programación, incluidos Arduino C/C++.
- La integración es fácil y no requiere instalación de software adicional, lo que hace que la plataforma sea accesible desde cualquier lugar con conexión a Internet.

4. Simulación Interactiva:

- La plataforma permite la simulación de comportamiento interactivo de tus proyectos, es decir, puedes ver el resultado del código que escribes en tiempo real en los componentes virtuales.
- Puedes controlar los dispositivos, como presionar botones virtuales o cambiar el valor de un potenciómetro, para observar cómo responden en la simulación.

5. Biblioteca de Componentes:

- Wokwi tiene una amplia biblioteca de componentes, que incluye desde los básicos (resistencias, LEDs, motores) hasta componentes más complejos como pantallas LCD, pantallas OLED, sensores de temperatura, y módulos de comunicación como el HC-05 (Bluetooth) o ESP32 Wi-Fi/Bluetooth.
- También soporta módulos de sonido, sensores de temperatura y otros dispositivos que permiten crear proyectos interactivos más avanzados.

6. Facilidad de Uso:

- La interfaz de usuario es intuitiva y fácil de usar, lo que permite a los usuarios arrastrar y soltar componentes para crear circuitos. No es necesario tener experiencia previa en diseño de circuitos electrónicos o programación para empezar.
- La plataforma también incluye ejemplos y plantillas de proyectos que puedes explorar para aprender y modificar según tus necesidades.

7. Visualización del Código:

- La plataforma permite ver cómo el código afecta al comportamiento de los componentes, lo que es útil para la depuración y aprendizaje. Si algo no funciona como se espera, puedes ver en tiempo real los valores de las variables y el estado de los pines.

8. Proyectos y Comunidad:

- Puedes guardar, compartir y colaborar en proyectos con otras personas a través de la plataforma. Esto facilita el trabajo en equipo y el intercambio de ideas.
- Además, hay una comunidad activa donde puedes buscar ejemplos de proyectos y obtener ayuda de otros usuarios.

9. Uso de Proyectos Reales:

- Wokwi permite simular proyectos reales y probar diferentes configuraciones de circuitos y código sin necesidad de hardware físico. Esto es ideal para prototipar ideas rápidamente antes de construir el proyecto real.

¿Cómo se usa Wokwi?

1. Acceder a la plataforma:
Solo necesitas una conexión a Internet y un navegador web para acceder a Wokwi. No es necesario instalar ningún software adicional.
2. Crear un nuevo proyecto:
Una vez dentro de la plataforma, puedes crear un nuevo proyecto. Puedes elegir entre varios microcontroladores como Arduino o ESP32, y luego comenzar a agregar componentes al diseño del circuito.
3. Programar el código:
Una vez que hayas configurado el circuito, puedes escribir el código directamente en el editor de Wokwi. La plataforma soporta el lenguaje de programación de Arduino IDE, por lo que puedes usar las mismas bibliotecas y funciones.
4. Simulación:
Después de escribir el código, puedes ejecutar la simulación. Wokwi te permite ver cómo se comporta tu circuito y cómo responde a tu código en tiempo real.
5. Compartir proyectos:
Puedes compartir tus proyectos con otros usuarios o exportarlos para su uso en otras plataformas.

2.5 VS Code con PlatformIO y Arduino Framework

Visual Studio Code (VS Code) es un editor de código fuente potente y ampliamente utilizado por desarrolladores debido a su flexibilidad, extensibilidad y compatibilidad con una variedad de herramientas y frameworks. En este proyecto, se emplea junto con PlatformIO, un entorno de desarrollo integrado (IDE) especializado en el desarrollo de sistemas embebidos, y el framework Arduino para facilitar la programación del ESP32. (PlatformIO, 2014)

Descripción

- VS Code: Es un editor de código fuente ligero que admite la instalación de extensiones para ampliar sus funcionalidades. Su diseño modular permite que sea compatible con una amplia gama de lenguajes de programación y herramientas.
- PlatformIO: Es una extensión de VS Code que proporciona un entorno IDE completo para el desarrollo de software en sistemas embebidos. Ofrece compatibilidad con múltiples microcontroladores, incluyendo el ESP32, y herramientas integradas para depuración, pruebas y administración de dependencias.

- Arduino Framework: Es un conjunto de bibliotecas y herramientas que simplifican la programación de microcontroladores. Su integración con PlatformIO permite trabajar con el ESP32 de manera eficiente y flexible, utilizando un lenguaje similar a C++.

Implementación

1. Instalación de VS Code y PlatformIO:
 - Descargar e instalar [Visual Studio Code](#).
 - Agregar la extensión PlatformIO IDE desde el marketplace de VS Code.
2. Configuración del entorno:
 - Crear un nuevo proyecto en PlatformIO seleccionando el ESP32 como placa objetivo.
 - Configurar el framework Arduino en el archivo platformio.ini.
3. Ventajas del uso conjunto:
 - Permite gestionar dependencias y bibliotecas de manera eficiente.
 - Facilita la depuración y monitoreo en tiempo real.
 - Compatible con herramientas avanzadas como terminales serie integradas y análisis de código.

Esta integración se utiliza para desarrollar el código que controla los motores DC. La facilidad de uso de Arduino Framework, combinada con las herramientas avanzadas de PlatformIO, permite una programación optimizada y eficiente para el ESP32. (CODE, s.f.)

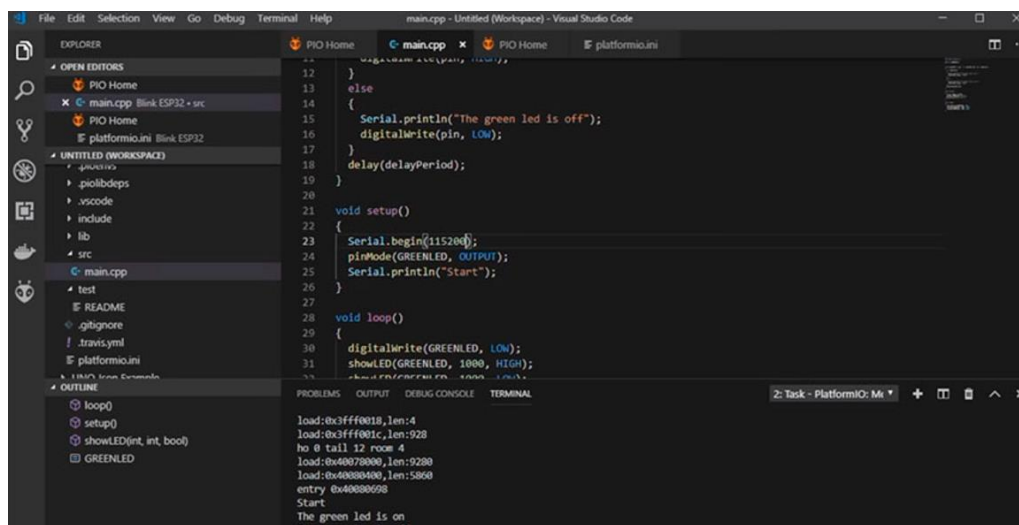


Figura 11 Pantalla de VS Code con PlatformIO para el ESP32
(Geek, 2005-2024)

2.6 Drivers de Espressif

Los drivers proporcionados por Espressif Systems son componentes de software esenciales para la comunicación y control del hardware en los microcontroladores ESP32. Estos controladores permiten que el hardware funcione de manera eficiente con el software al habilitar funciones específicas como la comunicación serial, el control de pines GPIO, PWM, y la conectividad Wi-Fi y Bluetooth.

Descripción

¿Qué son los drivers de Espressif?

Los drivers son módulos de software que actúan como intermediarios entre el hardware y las aplicaciones de alto nivel. En el caso del ESP32, Espressif ofrece una variedad de drivers que se integran con el ESP-IDF (Espressif IoT Development Framework) para facilitar el desarrollo de aplicaciones complejas.

1. Control de GPIO: Configuración y uso de los pines de entrada/salida (digitales y analógicos).
2. Comunicación serial: Proporcionan soporte para protocolos como UART, SPI e I2C.
3. PWM (Pulse Width Modulation): Control de velocidad de motores, brillo de LEDs y otras aplicaciones similares.
4. Conectividad: Drivers para la gestión de Wi-Fi y Bluetooth, habilitando conexiones inalámbricas robustas.
5. ADC/DAC: Soporte para conversión analógica-digital y digital-analógica.

(ESPRESSIF, 2024)

Implementación

1. Instalación de los drivers:
 - Los drivers se incluyen en el ESP-IDF, que puede instalarse desde el sitio oficial de Espressif.
 - Al trabajar con Arduino Framework en PlatformIO, estos drivers están preconfigurados y listos para usarse.
2. Configuración básica:
 - Configurar los pines GPIO o periféricos específicos en el código mediante las funciones de los drivers.
 - Integrar la funcionalidad necesaria, como lectura de sensores o manejo de motores, utilizando los módulos correspondientes.

3. Uso práctico en el proyecto:

- Los drivers permiten leer valores analógicos del potenciómetro, generar señales PWM para los motores, y controlar la dirección de los mismos a través de los pines GPIO del ESP32.
- Facilitan el envío y recepción de datos mediante protocolos UART para monitorear el sistema.

Los drivers son esenciales para implementar funciones como:

- Control de motores DC: Generación de señales PWM y manejo de la polaridad del puente H.
- Lectura de entradas analógicas: Uso del ADC para interpretar las señales del potenciómetro.
- Conectividad: Potencial uso de Wi-Fi o Bluetooth para monitorear el sistema de manera remota en futuras expansiones del proyecto.

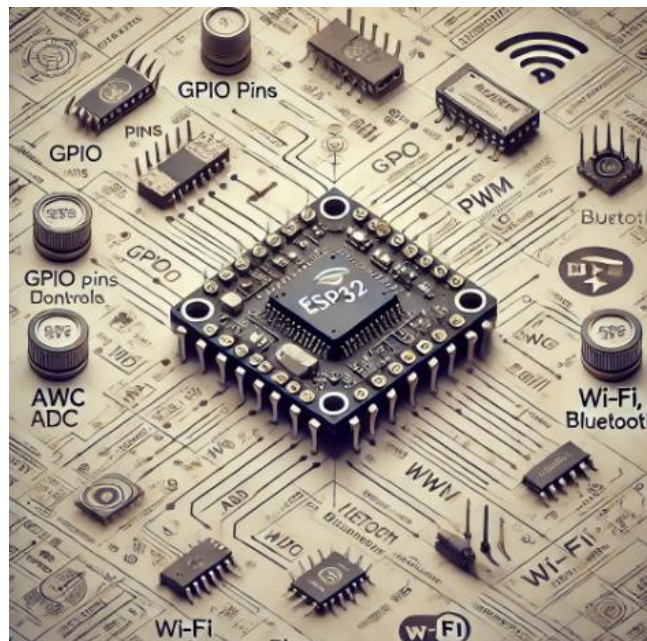


Figura 12 ESP 32 componentes
(TuEquipoSeo, 2014)

2.7 Dip Switch

Un Dip Switch es un interruptor de baja potencia utilizado para realizar configuraciones en circuitos electrónicos y sistemas. Este componente es ampliamente utilizado para la selección manual de opciones o ajustes de configuración sin necesidad de utilizar software. Los Dip Switches son comunes en aplicaciones como la configuración de dispositivos electrónicos, como impresoras, routers, sistemas embebidos, y proyectos de automatización.

¿Qué es un Dip Switch?

Un Dip Switch (Dual In-line Package Switch) es un conjunto de interruptores pequeños que se montan en una placa de circuito impreso (PCB). Estos interruptores permiten activar o desactivar funciones específicas al conectarlos a tierra o a voltaje positivo. Los Dip Switches suelen estar formados por varios interruptores en línea, y cada uno puede tener dos posiciones: ON (encendido) o OFF (apagado).

Funcionamiento:

- Cada interruptor del Dip Switch corresponde a una señal binaria: ON para 1 y OFF para 0.
- En sistemas digitales, se usan para ajustar parámetros de configuración, como la dirección de comunicación, el modo de operación, o la configuración de hardware.
- En proyectos de automatización, el Dip Switch se utiliza para cambiar el comportamiento de dispositivos o microcontroladores sin la necesidad de cambiar el software. (Vasco, s.f.)

Implementación

1. Instalación de un Dip Switch:
 - Se conecta en el circuito como un conjunto de pines, generalmente con una configuración de 2, 4, 6 u 8 interruptores.
 - Cada interruptor controla una función específica, ya sea para cambiar el modo de un dispositivo o para ajustar parámetros operativos del sistema.
2. Configuración:
 - El Dip Switch se utiliza para cambiar entre diferentes configuraciones de hardware, como la dirección de un motor, la velocidad de funcionamiento o la selección de un puerto de comunicación.
 - A través de un código o sistema de lógica digital, los valores "ON" y "OFF" de los interruptores se interpretan como valores binarios para realizar cambios en el sistema.
3. Uso en el Proyecto:
 - En este proyecto, un Dip Switch puede usarse para seleccionar diferentes modos de operación del sistema de control de los motores, como por ejemplo: activación de la dirección de giro o encendido/apagado de los motores.

Los Dip Switches permiten realizar ajustes manuales para el control de los motores DC.

- Control de dirección: Configuración de la dirección de rotación de los motores, donde un interruptor puede cambiar la polaridad.
- Selección de velocidad o modos de operación: Cambiar entre diferentes velocidades de los motores o incluso configurar el modo de operación (avanzar, retroceder, detenerse).

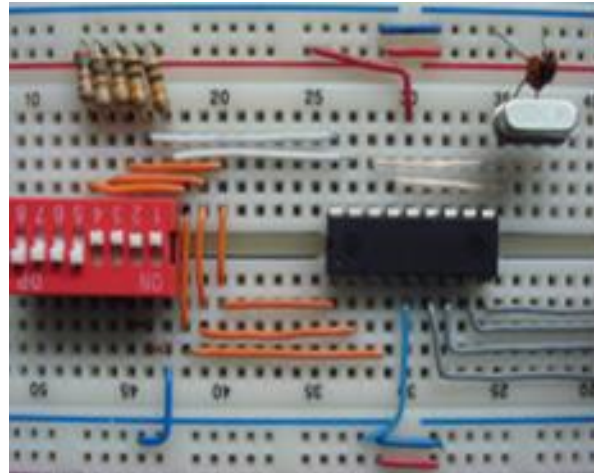


Figura 13 Dip Switch en circuito
(AG, 2021)

2.8 Resistencias

Las resistencias son componentes fundamentales en circuitos electrónicos, cuyo principal objetivo es limitar el flujo de corriente eléctrica. Actúan como una barrera para controlar la cantidad de corriente que pasa a través de un circuito, protegiendo otros componentes de sobrecargas y proporcionando condiciones estables para su funcionamiento. Son componentes pasivos que cumplen un papel clave en la creación de circuitos funcionales y seguros. (CORPORATION, 2021)

¿Qué es una resistencia?

Una resistencia es un componente electrónico que resiste el paso de la corriente eléctrica. Su valor se mide en ohmios (Ω), y su función principal es limitar o regular la cantidad de corriente que circula por un circuito. Este componente está fabricado principalmente de materiales como carbono, metal o películas metálicas que presentan una resistencia eléctrica inherente al flujo de corriente.

Ley de Ohm:

La Ley de Ohm describe la relación entre la tensión (V), la corriente (I) y la resistencia (R) en un circuito:

$$V=I \times R \quad R=V \div I \quad I=V \div R$$

- V: Voltaje en voltios (V)
- I: Corriente en amperios (A)
- R: Resistencia en ohmios (Ω)

(ppeeenergy, 2024)

Implementación

1. Selección de resistencias:
 - El valor de la resistencia se determina en función de la cantidad de corriente que se desea limitar en el circuito. Las resistencias deben elegirse según los requerimientos de voltaje y corriente para no exceder su capacidad de disipación de energía.
 - El valor se especifica mediante un código de colores o un número impreso en el cuerpo de la resistencia.
2. Colocación en el circuito:
 - Las resistencias pueden colocarse en serie o en paralelo dependiendo de la necesidad del circuito.
 - En serie: Se suman los valores de las resistencias.
 - En paralelo: Se calcula la resistencia equivalente utilizando la fórmula inversa

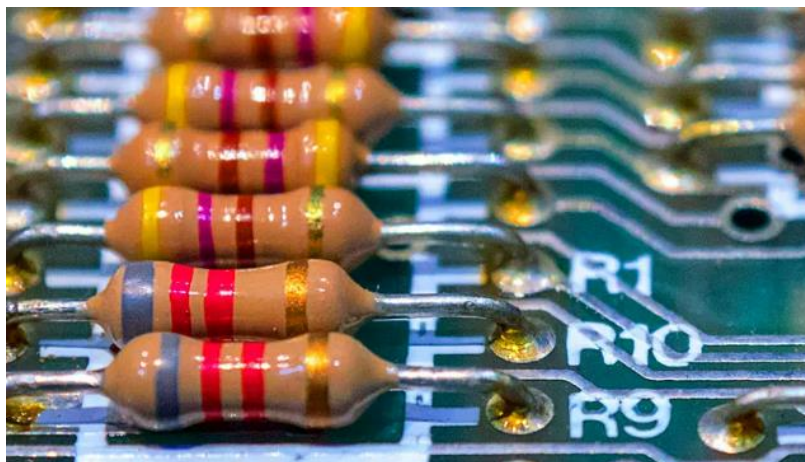


Figura 14 Resistencias en un circuito

(KeekNetic, 2002-2024)

2.9 Potenciómetro de 100k

Introducción

El potenciómetro es un componente electrónico utilizado para ajustar de manera manual el valor de resistencia en un circuito. Este dispositivo permite regular y controlar diferentes parámetros, como la velocidad de un motor, el volumen de un altavoz, o la intensidad de una luz, mediante el ajuste de su resistencia. El potenciómetro de 100k ohmios es una versión comúnmente utilizada en proyectos de electrónica, ideal para aplicaciones que requieren un control preciso sobre los circuitos de señal.

¿Qué es un potenciómetro?

Un potenciómetro es un tipo de resistencia variable que cuenta con un mecanismo rotatorio o deslizante para modificar su resistencia. Este dispositivo está compuesto por un resistido (material resistivo) y un contacto móvil que se desplaza a lo largo de él. Su valor de resistencia varía dependiendo de la posición del contacto móvil, permitiendo así el ajuste de la corriente que fluye por el circuito.

Características principales de un potenciómetro de 100k:

- Valor de resistencia: El valor nominal del potenciómetro de 100k es 100,000 ohmios. Este valor determina la capacidad de ajuste de resistencia en el circuito.
 - Tipo: Generalmente, los potenciómetros de 100k son de tipo lineales o logarítmicos, dependiendo de su aplicación (para ajustes de volumen, etc.).
 - Aplicaciones: Se usan comúnmente en circuitos analógicos para ajustar señales, en controles de volumen, o para crear divisores de voltaje.
- (Electrónica, 2017 - 2022)

Implementación

1. Selección del Potenciómetro de 100k:

El valor de 100k ohmios es adecuado para aplicaciones donde se necesita un rango más amplio de ajuste, como la regulación de velocidad en motores DC o la modulación de señales analógicas en sistemas de control.

2. Conexión en el Circuito:

En este proyecto, el potenciómetro de 100k se utiliza para regular la velocidad de los motores mediante un divisor de voltaje o para enviar señales analógicas al microcontrolador ESP32.

- Los tres pines del potenciómetro se conectan de la siguiente manera:
 - Un pin a voltaje de alimentación (Vcc).
 - El segundo pin al punto de referencia (el pin de lectura del ESP32 o ADC).
 - El tercer pin a tierra (GND).

3. Configuración en el Código:

En el código, el ESP32 lee la señal analógica del potenciómetro a través de su convertidor analógico-digital (ADC). La lectura de este valor se mapea para controlar la velocidad del motor o cualquier otra variable relevante.

(FÁCIL, 2024)

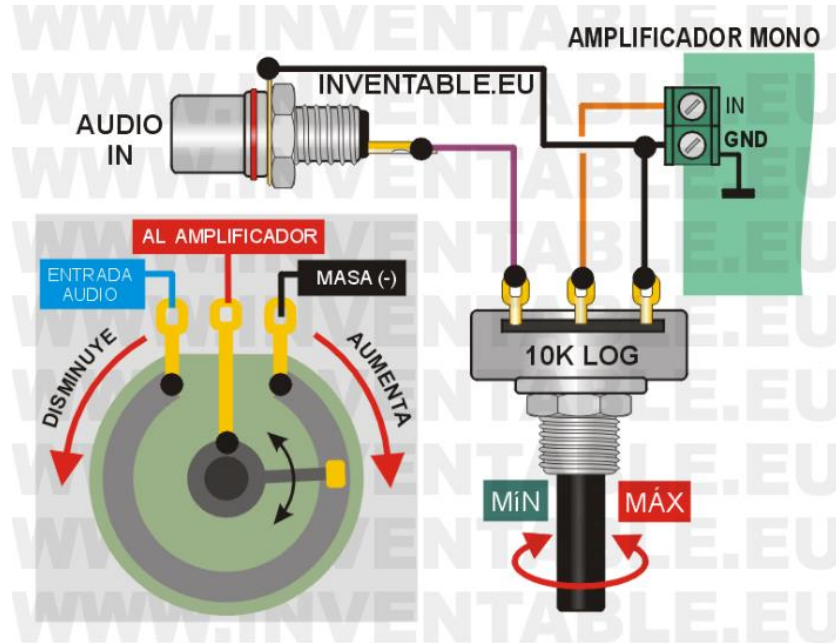


Figura 15 Potenciómetro estructura
(INVENTABLE.EU, 2010-2020)

CAPITULO 3. Desarrollo

3.1 CONFIGURACION DEL ENTORNO.

Se descarga el editor de código VSCode , se necesita tener Python instalado también , en este caso se descargará PlatformIO como extensión de VS Code en la tienda de aplicaciones , como se muestra a continuación, abrirá un menú contextual que solicitara abrir en la aplicación , una vez instalado es necesario reiniciar el editor:

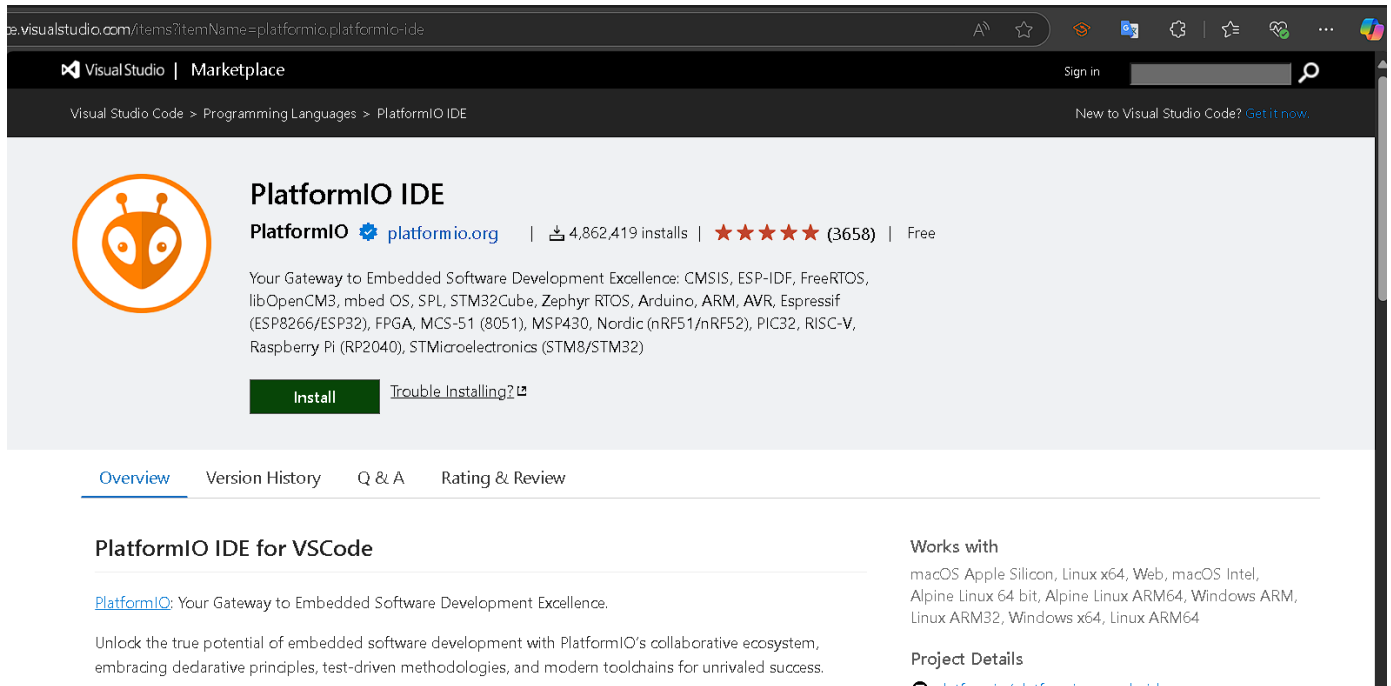


Figura 16 DESCARGA DE EXTENSION (platformio.org, 2024)

Se abre el editor de visual Studio Code y entonces nos cargara el siguiente menú:

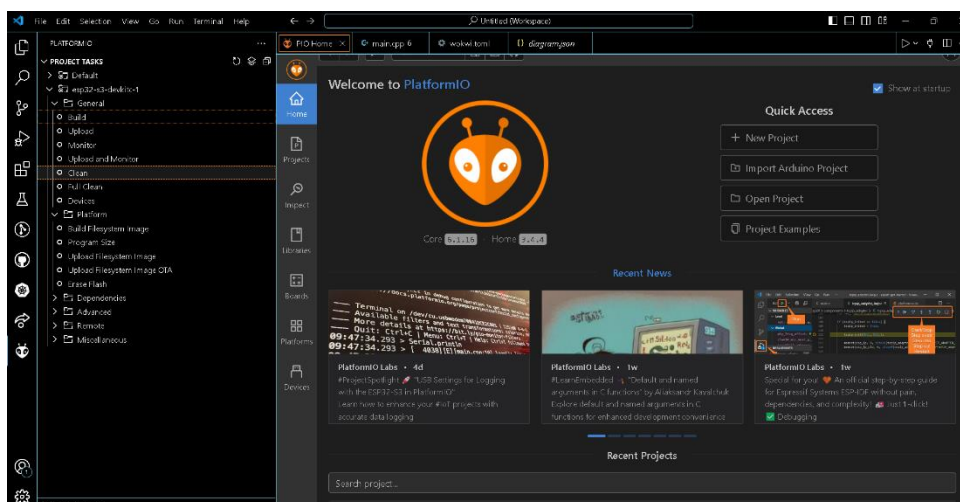


Figura 17 ITESS Equipo Menú de inicio de PlatformIO

En este caso es necesario instalar las herramientas de Espressif desde el editor se va al menú de Plataformas y se descarga la herramienta de Espressif:

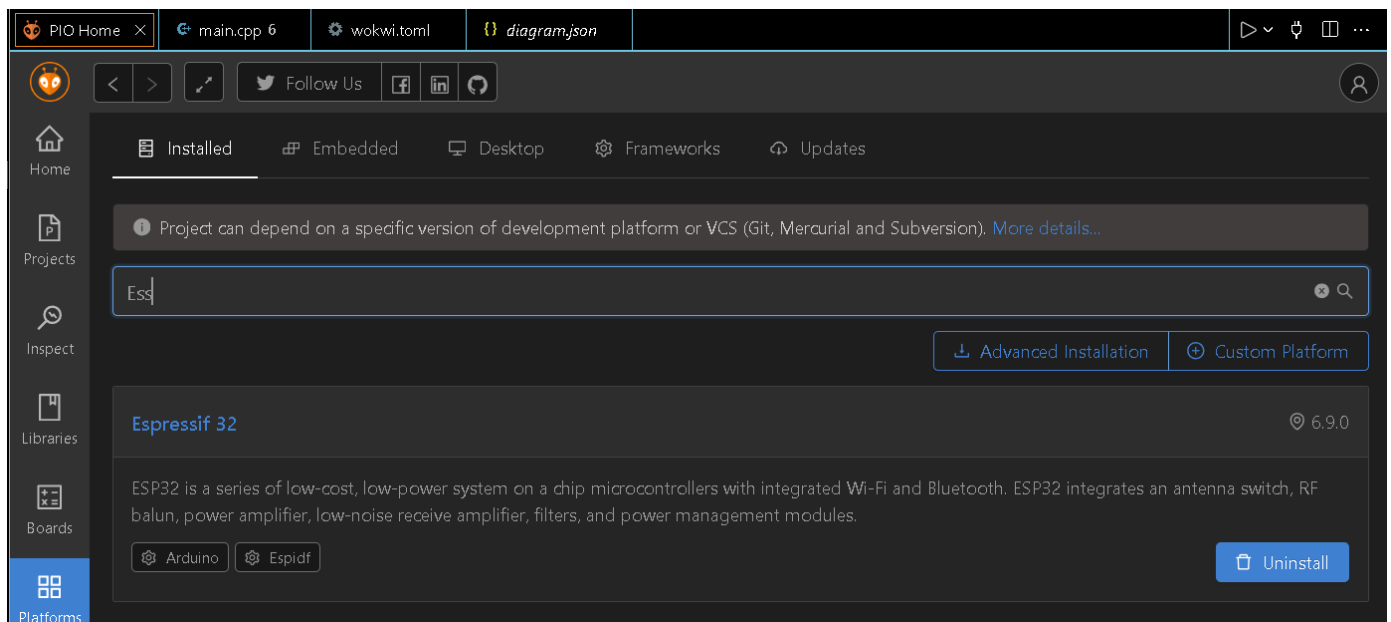


Figura 18 Espressif 32 Instalación de plataformas.

Después de instalarlo debe aparecer en el menú de la siguiente forma, lo que indica que el editor de PlatformIO lo detecta, y la instalación sucedió con éxito.

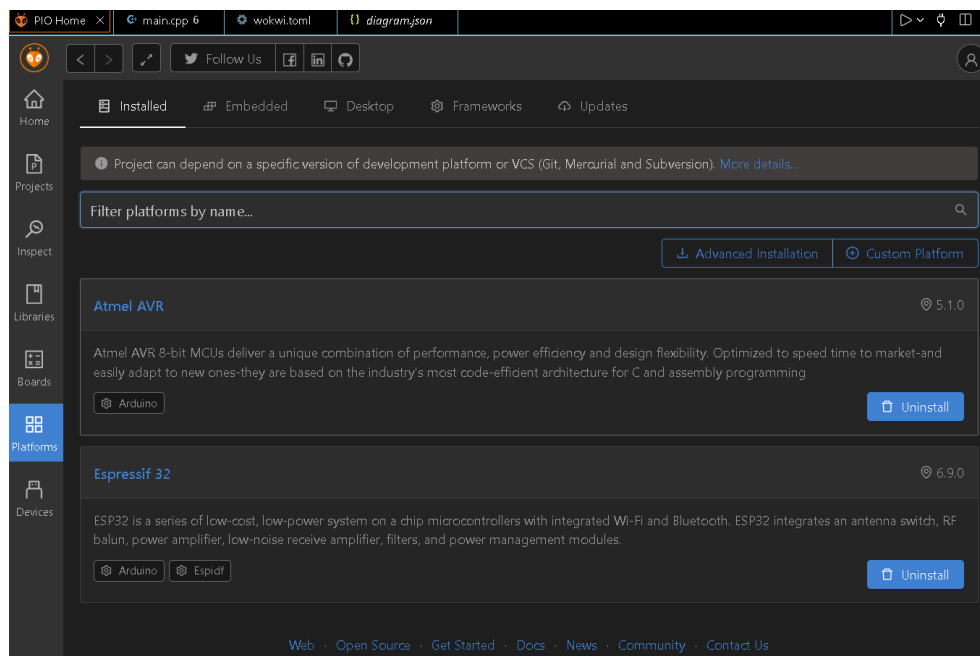


Figura 19 Plataformas instaladas dentro de PlatformIO

Configuración del archivo .ini del proyecto lo cual nos permitirá agregar librerías, editar la velocidad del ESP32, agregar el framework de Arduino para facilitar la programación de c++,

cabe aclarar que al momento de compilar el código no se compila desde visual Studio code como se haría con un simple archivo de c++ CPP, si no que en lugar de eso se accede al icono de Plataforma y se compila desde este , en el apartado de BUILD., si no aparece nuestro proyecto en este menú , se debe abrir desde el menú de Home donde podemos buscar nuestros proyectos.

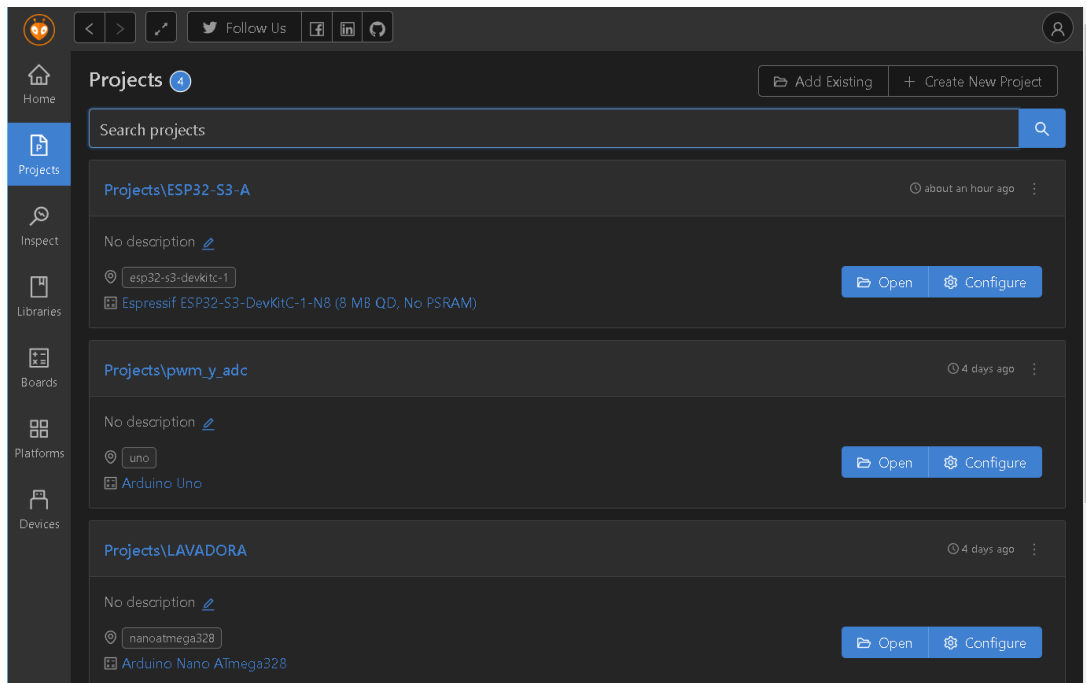


Figura 20 Proyectos abiertos.

Aquí tenemos el menú de compilación, y subida del programa a nuestro sistema embebido., nos muestra las opciones menu, upload, monitor, upload and monitor, clean: para limpiar el proyecto, full clean: en caso de generar errores esta opción limpia todo el proyecto y recompila todo .

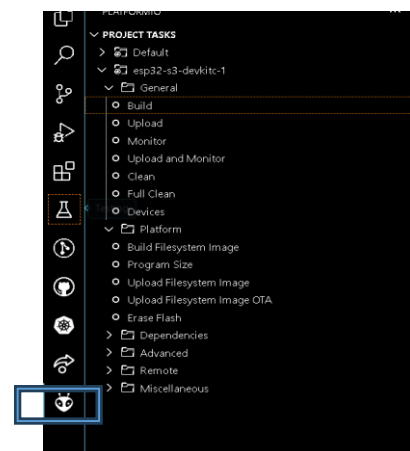


Figura 21 Menú de proyectos, aquí

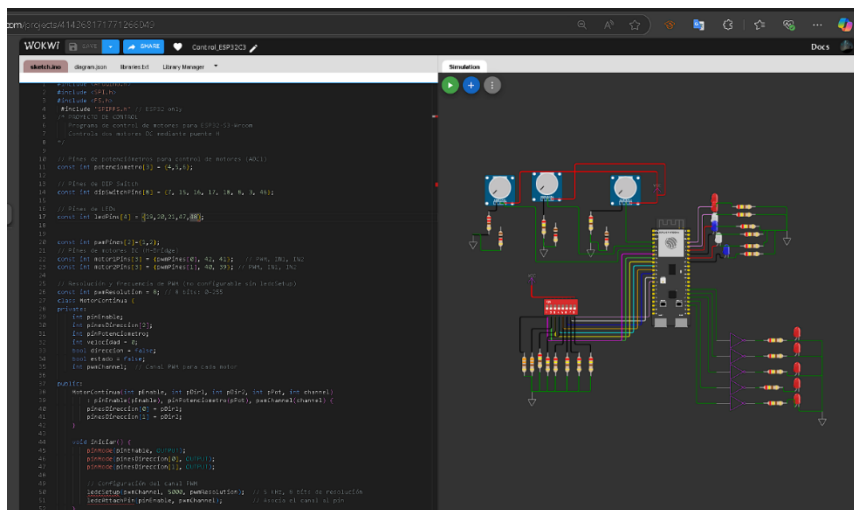


Figura 22 Edición del circuito en WOKWI

Ahora nos vamos a crear un proyecto en Wokwi y en esta plataforma podemos simular y/o diseñar en caliente(en el mismo editor online) , en caso de que tenga el editor online de las librerías que requerimos y los dispositivos , de lo contrario podemos usar otros simuladores como o es proteus o simul-IDE., que requieren configurar asignar el .hex con el que trabajara el simulador, si se trabaja online el simulador WOKWI mostrara la salida con el código en pantalla. En

caso de que no las detecte descargaremos el proyecto , para seguir trabajando offline con este , simse desea simular o ver el diseno offline es necesario configurar la version de comunidad , siguiendo los pasos que nos indica wokwi en su diocumentacion.

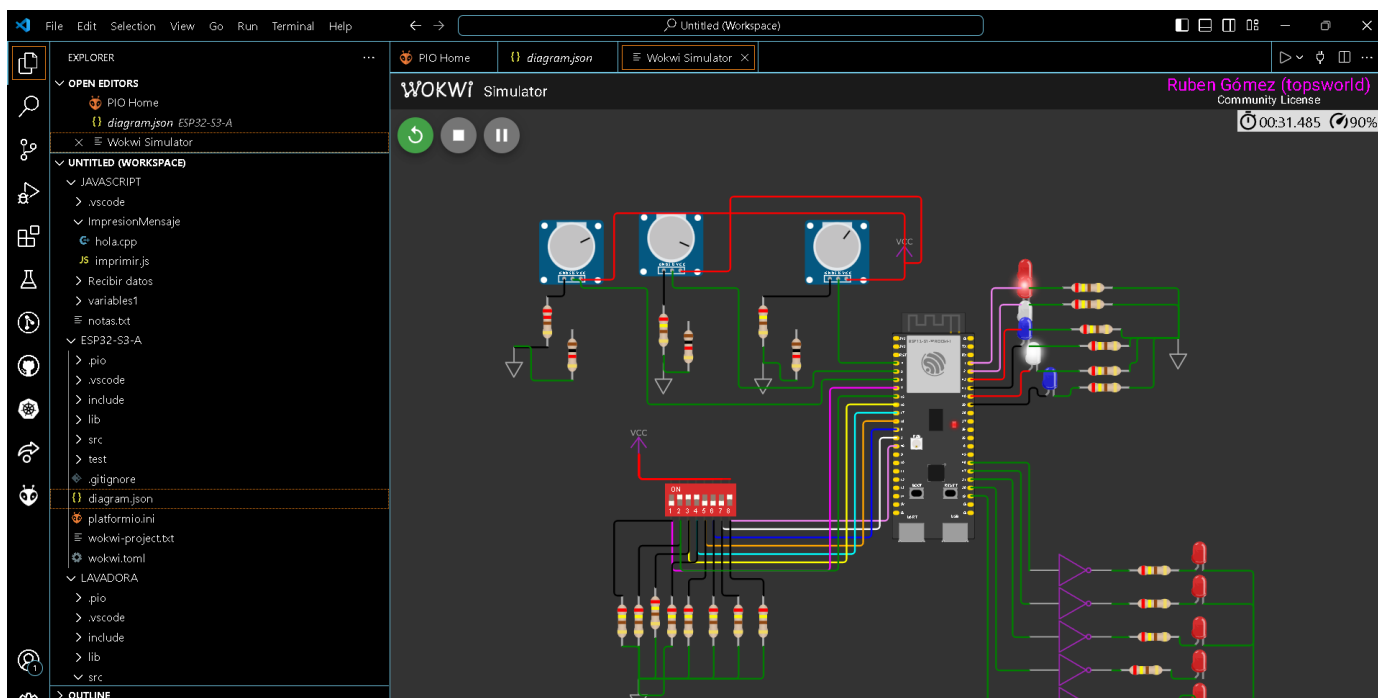


Figura 23 Captura de pantalla del proyecto en VS code offline

Configuración del archivo. toml para este proyecto , se debe hacer bien o la simulación no cargara , en la documentación de WOOKWI nos dicen como depende la placa.



Figura 24 toml para simulación

Configuración del de .ini en esta cabecera se incluye entre corchetes el proyecto, se escribe la plataforma en la que incluyen los controladores, la placa(el tipo de esp32 con la que es compatible el ESP32-S3-WROOM-1), se usa el framework de Arduino y se configura velocidad del monitor serie, también se deben incluir librerías si se usan.

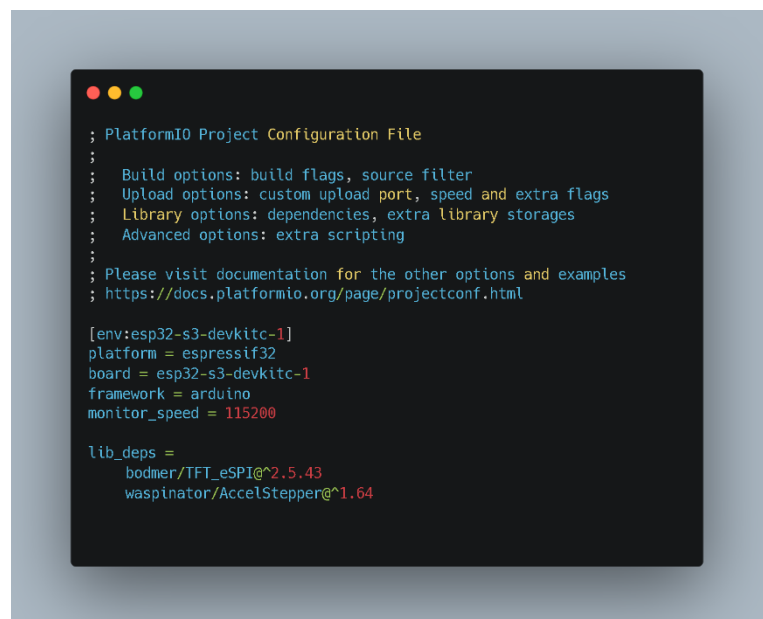


Figura 25 Plataformio.ini

3.2 Circuito de conexiones de la parte de lógica y control (ESP32)

Lo siguiente en el proceso fue realizar el circuito en físico, aquí se realizaron las conexiones para el puente H , se debe recordar que las GND ,se juntan para un correcto funcionamiento del circuito(los positivos NO), el diagrama es el que está en las capturas del simulador, solo que en la protoboard, se sueldan los motores con cable para poder conectar sus terminales a las terminales del módulo puente H , se usó un voltaje de 17V para el pin de alimentación de los motores del puente H.

Primero se tienen que montar dos placas de prototipos para poder abarcar al esp'32, puesto que este es algo grande se le quita una pista cada lado de las dos placas perforadas donde poseen la pistas de alimentación de los positivos y negativos, y se conectan entre si en los lados . las pistas de alimentación son las que tiene franjas azul y roja y están en serie (a veces si es china esta dividida en dos esta línea , en ese caso en las que si vanes necesario crear un pequeño puente ,estas están destinadas a la alimentación) . de la forma e que se indica la siguiente figura.

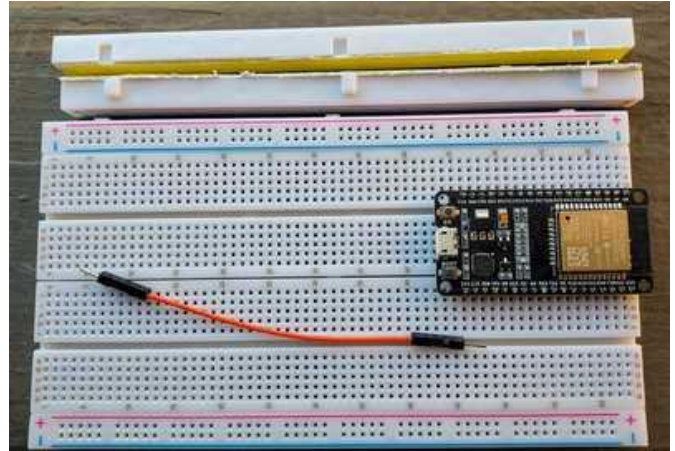


Figura 26 colocación en protoboard (jBo)

Una vez hecho esto procederemos a realizar las conexiones, en caso de que el dispositivo con el que trabaje no sea el ESP32-s3-wroom-1 o algún otro modelo con características similares en especial en el pinout y haya modificado el programa deberá adaptarlo a su código modificado. En nuestro caso, nosotros lo conectamos de la siguiente forma:

```
// Pines de potenciómetros para control de motores (ADC1)
const int potencio[3] = {4,5,6};

// Pines de DIP Switch
const int dipSwitchPins[8] = {7, 15, 16, 17, 18, 8, 3, 46};

// Pines de LEDs
const int ledPins[4] = {19,20,21,47};

const int pwmPines[2]={1,2};
// Pines de motores DC (H-Bridge)
const int motor1Pins[3] = {pwmPines[0], 42, 41}; // PWM, IN1, IN2
const int motor2Pins[3] = {pwmPines[1], 40, 39}; // PWM, IN1, IN2
```

Figura 27 Fragmento que representa configuración de pines

En la cabecera del código que se encuentra aquí se encuentran en la cabecera definidos en arreglo las direcciones, o pines donde se encuentran los pines y el uso que se les dará, utilizando estrategias de código auto comentado como se vio en la materia de programación.

Aquí se definen los tipos de pines y el uso, por ejemplo los pines a usar del timo pwm, que son los encargados de mandar señal al pin enable del modulo controlador puente H, y permitirán usar el pwm como método de control de velocidad de motores al variar el voltaje, variando el periodo de encendido de este modulo para este motor, variando así su velocidad lo que se conoce por variación de velocidad por variación de voltaje, con la ventaja de ser un sistema basado en programación y que esta función puede ser extendida, en especial por el enfoque orientado a objetos y orientado a método.

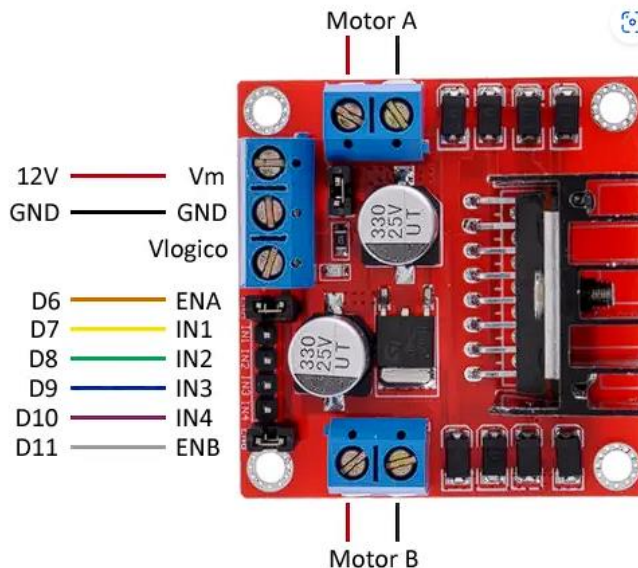


Figura 28 esquema de conexión del puente H

Las conexiones entre el microcontrolador están definidas en esta parte del código anterior, en el caso de los pines pwm, simplemente es el elemento cero del arreglo es 1 (ENA), y el elemento uno del arreglo es el pin 2 (ENB), IN1= pin 42, IN2= pin 41, IN3= 40, IN4= 39.

```
const int pwmPines[2]={1,2};
// Pines de motores DC (H-Bridge)
const int motor1Pins[3] = {pwmPines[0], 42, 41}; // PWM, IN1, IN2
const int motor2Pins[3] = {pwmPines[1], 40, 39}; // PWM, IN1, IN2
```

Figura 29 Conexiones de pines que son correspondientes al motor



Figura 30 conf. de resistencias salida al modulo

En esta parte hay que tener cuidado se debe colocar las resistencias así y luego los leds, para visualizar las señales, de esa forma encendrán los leds, y la salida que va como señal al módulo, debe ser captada antes de entrar a la resistencia, de tal modo que esta actúe como resistencia pull down y señal visualizadora.

Ejemplo en proteus de función

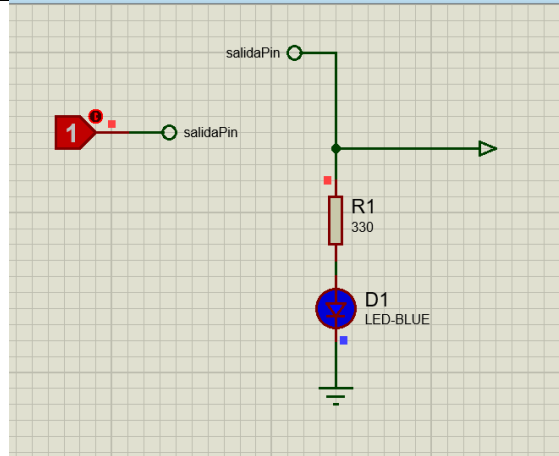


Figura 31 pull down activo señal alta

Ejemplo de funcionalidad

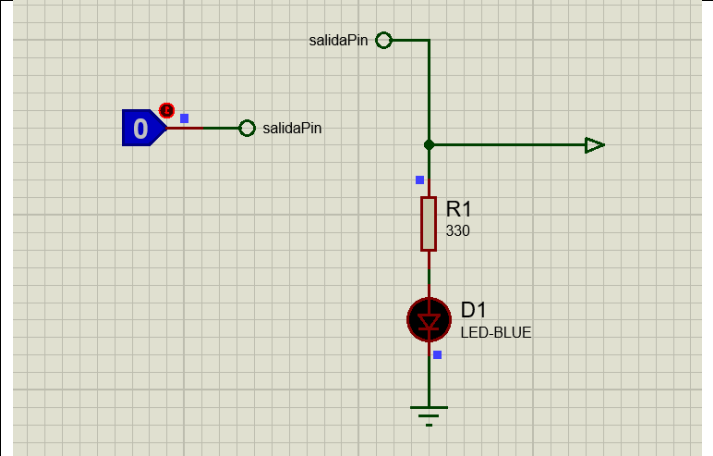


Figura 32 pull down inactivo señal baja

Las conexiones del switch también son del tipo pulldown: y se pueden ver en el circuito como están configuradas, esto con el objetivo de que si el switch cambia de estado el microcontrolador puede comparar correctamente el estado y cambiarlo a fin de que el programa no genere errores, si el switch está mal conectado puede generar falsos contactos, este debe hacer contacto correcto con la protoboard.

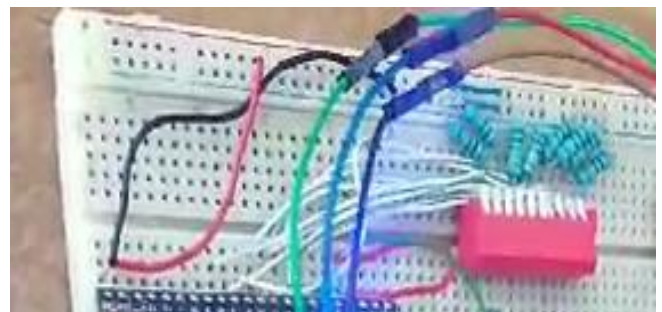


Figura 33 configuración del switch en pull down

Si el microcontrolador lo permite se pueden declarar como pulldown, s no basta con dejarlo declarado como INPUT dentro del código del programa, en nuestro caso si lo configuramos de esa forma.

```
void configurarPines() {  
    // Inicializar motores  
    /*RESTO DE CONFIGURACION*/  
    // Configurar pines de DIP Switch  
    for (int pin : dipSwitchPins) {  
        pinMode(pin, INPUT_PULLDOWN);  
    }  
}
```

Figura 34 Método ConfigurarPines(), entradas tipo INPUT_PULLDOWN

3.3 Montaje del sistema de potencia con Puente H.

La GND comun en nuestro caso fue necesaria para el correctofuncionamiento del prototipo de control de velocidad , esto jntado unicamente los negativos del circuito de control que operaba con senales digitales, y el de potencia que controlo voltajes de 12 a 24v, esto mostrandolo al profesor . Se realizan las connexiones con GND comun , las conexiones de señal y el ensamble

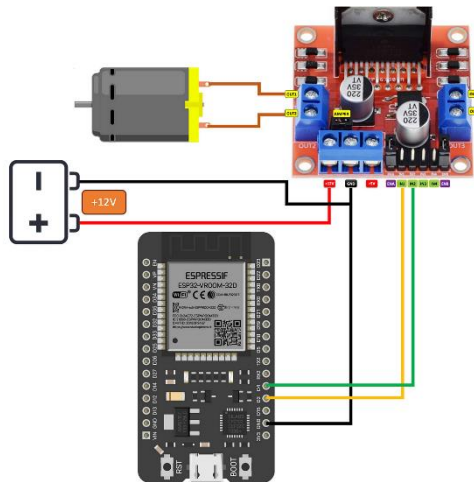


Figura 35 Conexión (Medium)GND común ejemplo de los motores siguiendo el diagrama de conexión del puente H.

TABLA DE EVIDENCIAS de montaje.

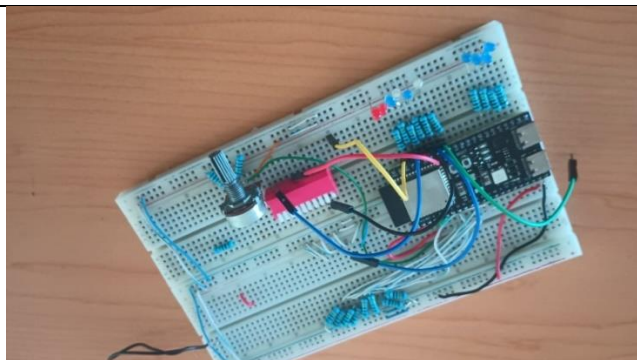


Figura 36 Progreso de circuito

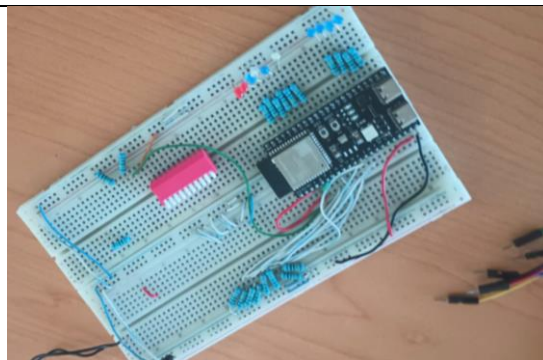


Figura 37 Progreso de circuito

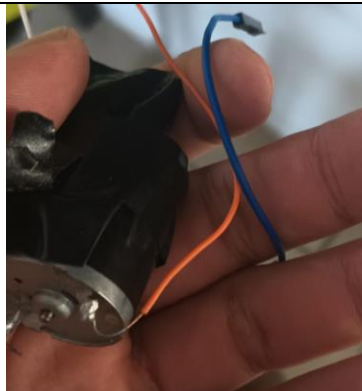


Figura 38 motor soldadura de pines



Figura 39 soldadura de pines y protección con cinta de aislar

3.4 Compilación y programación del sistema, ya diagramado.

Al conectar el esp32-s3-wroom con el ordenador el administrador de dispositivos debe reconocerlo, si no lo reconoce hay varias opciones, se instalaron mal los controladores espressif , el esp32 es chino y tiene algún controlador genérico y se debe instalar, esto instalando os mas comunes, o checando si el fabricante/proveedor lo proporciona en la página donde se adquirió, o si nos proporciona información de que driver ,es .

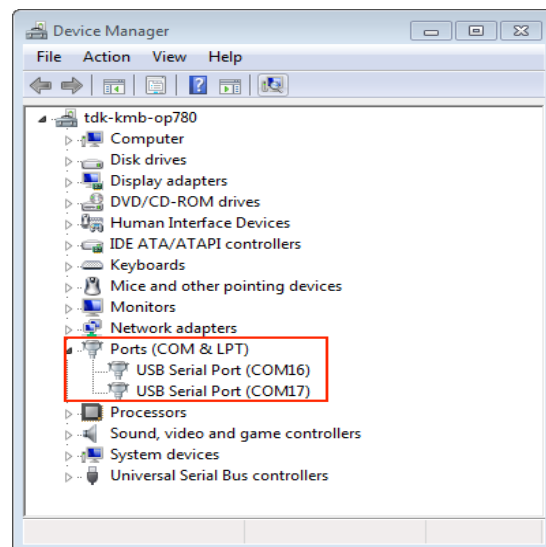


Figura 40 Administrador de dispositivos
(ESPRESSIF)

La conexión es bastante simple es el puerto USB del ESP32-s3con algún puerto de nuestro ordenador :

Visualización conexión PC- ESP32

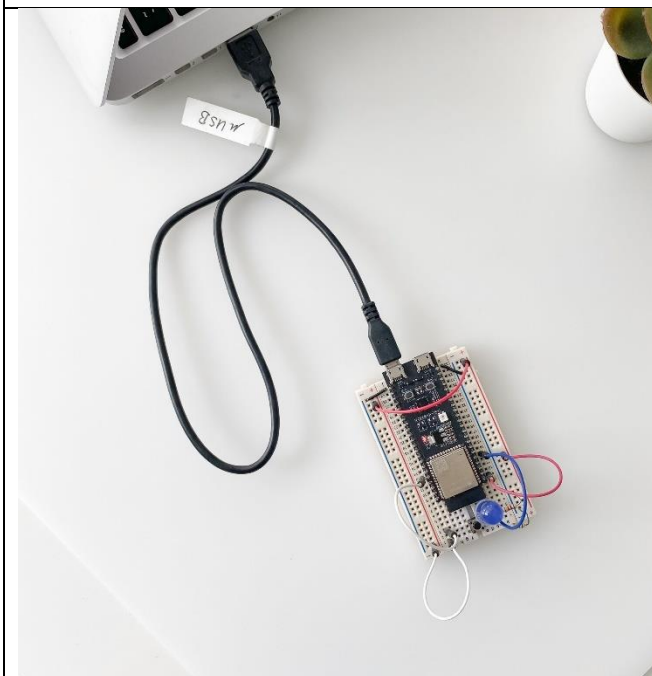


Figura 41 Conexión del computador (hutscape.com)

EL puerto que se usa es el USB serial

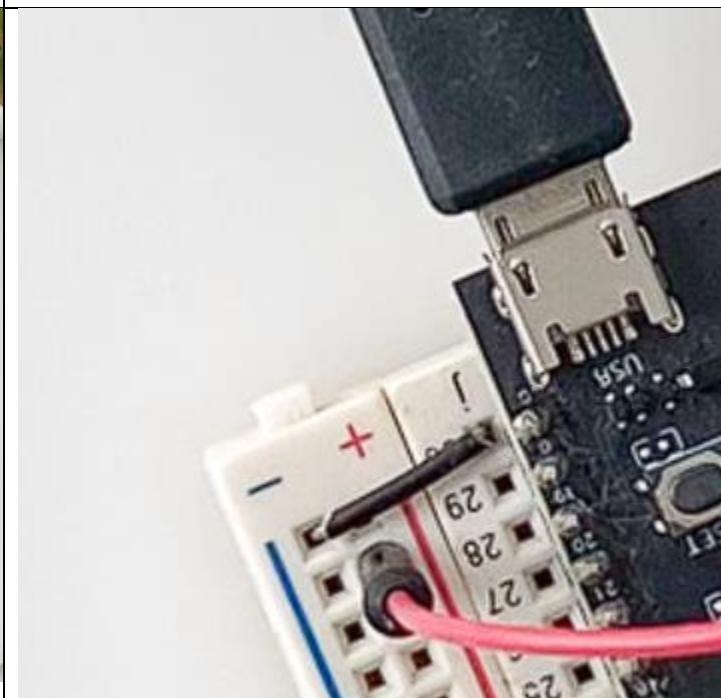


Figura 42 USB serial

Ya conectado el ESP32 compilamos y subimos el PROYECTO. Para subir el proyecto a veces es necesario instalar drivers para esp32 genéricos como CH340, le damos en subir y el IDE detectara el puerto COM, en algunos modelos del ESP32 es necesario presionar un botón mientras se carga el firmware.

```
- framework-arduinoespressif32 @ 3.20017.0 (2.0.17)
- tool-esptoolpy @ 1.40501.0 (4.5.1)
- toolchain-riscv32-esp @ 8.4.0+2021r2-patch5
- toolchain-xtensa-esp32s3 @ 8.4.0+2021r2-patch5
LDF: Library Dependency Finder -> https://bit.ly/configure-pio-ldf
LDF Modes: Finder ~ chain, Compatibility ~ soft
Found 35 compatible libraries
Scanning dependencies...
Dependency Graph
|-- TFT_eSPI @ 2.5.43
|-- AccelStepper @ 1.64.0
|-- SPIFFS @ 2.0.0
|-- FS @ 2.0.0
|-- SPI @ 2.0.0
Building in debug mode
Retrieving maximum program size .pio\build\esp32-s3-devkitc-1\firmware.elf
Checking size .pio\build\esp32-s3-devkitc-1\firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [ = ] 5.9% (used 19368 bytes from 327680 bytes)
Flash: [ = ] 8.7% (used 290909 bytes from 3342336 bytes)
===== [SUCCESS] Took 3.02 seconds =====
* Terminal will be reused by tasks, press any key to close it.
```

Figura 43 mensaje de compilación exitosa , propietario

Si no muestra el SUCESS ocurrió un error y hay que corregirlo , para eso es útil la salida de la terminal integrada de PowerShell de VS code.

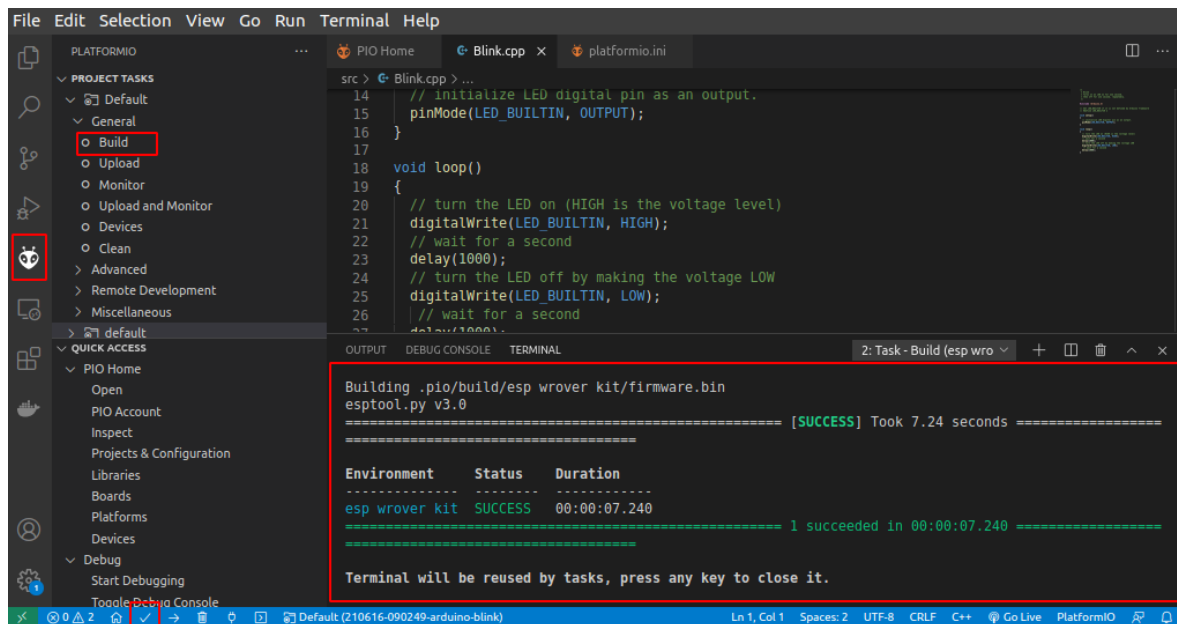


Figura 44 MENSAJE de programación de placa exitoso (Bassi, 2021)

Código

```
#include <Arduino.h>

#include <SPI.h>

#include <FS.h>

#include "SPIFFS.h" // ESP32 only

/* PROYECTO DE CONTROL

Programa de control de motores para ESP32-S3-Wroom

Controla dos motores DC mediante puente H

*/

// Pines de potenciómetros para control de motores (ADC1)
const int potenciometro[3] = {4,5,6};

// Pines de DIP Switch
const int dipSwitchPins[8] = {7, 15, 16, 17, 18, 8, 3, 46};

// Pines de LEDs
const int ledPins[4] = {19,20,21,47};

const int pwmPines[2]={1,2};

// Pines de motores DC (H-Bridge)
const int motor1Pins[3] = {pwmPines[0], 42, 41}; // PWM, IN1, IN2
const int motor2Pins[3] = {pwmPines[1], 40, 39}; // PWM, IN1, IN2

// Resolución y frecuencia de PWM (no configurable sin ledcSetup)
const int pwmResolution = 8; // 8 bits: 0-255

class MotorContinua {
private:
    int pinEnable;
```

```

int pinesDireccion[2];
int pinPotenciometro;
int velocidad = 0;
bool direccion = false;
bool estado = false;
int pwmChannel; // Canal PWM para cada motor
//bool direccionAntigua=false,nuevaDireccion=false;

public:
    MotorContinua(int pEnable, int pDir1, int pDir2, int pPot, int channel)
        : pinEnable(pEnable), pinPotenciometro(pPot), pwmChannel(channel) {
        pinesDireccion[0] = pDir1;
        pinesDireccion[1] = pDir2;
    }

    void iniciar() {
        pinMode(pinEnable, OUTPUT);
        pinMode(pinesDireccion[0], OUTPUT);
        pinMode(pinesDireccion[1], OUTPUT);

        // Configuración del canal PWM
        ledcSetup(pwmChannel, 5000, pwmResolution); // 5 kHz, 8 bits de resolución
        ledcAttachPin(pinEnable, pwmChannel); // Asocia el canal al pin
    }

    void setVelocidad() {
        velocidad = constrain(lerPpotenciometro(pinPotenciometro), 0, 255);
        ledcWrite(pwmChannel, velocidad); // Control de la velocidad con PWM
    }

    void encender() {

```



```

        estado = true;

        actualizarDireccion();

        setVelocidad();
    }

    void apagar() {
        estado = false;

        digitalWrite(pinesDireccion[0], LOW);

        digitalWrite(pinesDireccion[1], LOW);

        ledcWrite(pwmChannel, 0); // Apagar el PWM
    }

    void cambiarDireccion(bool dir) {
        if(dir != direccion){
            apagadoTemporal();//solo se detiene un tiempo si hay cambio de direccion
        }

        direccion = dir;

        actualizarDireccion();//se actualiza direccion
    }

    int getVelocidad(){
        return velocidad;
    }

private:
    int leerPotenciometro(int pinPot) {
        int valorPotenciometro = analogRead(pinPot); // Lectura del ADC

        return map(valorPotenciometro, 0, 4095, 0, 255); // Mapear a rango de 0-255
    }

```

```

void apagadoTemporal() {
    //Esto se realiza para evitar saltos del motor
    //al cambiar bruscamente de velocidad
    //no se modifica el state porque solo
    //es un estado de transición
    digitalWrite(pinesDireccion[0], LOW);
    digitalWrite(pinesDireccion[1], LOW);
    ledcWrite(pwmChannel, 0); // Apagar el PWM
    delay(250); //delay pequeño
}

void actualizarDireccion() {
    if (estado) {
        if (direccion) {
            digitalWrite(pinesDireccion[0], HIGH);
            digitalWrite(pinesDireccion[1], LOW);
        } else {
            digitalWrite(pinesDireccion[0], LOW);
            digitalWrite(pinesDireccion[1], HIGH);
        }
    }
}

};

// Creación de instancias de motores
// Creación de instancias de motores con diferentes canales PWM
MotorContinua motor1(motor1Pins[0], motor1Pins[1], motor1Pins[2], potenciometro[0], 0);
MotorContinua motor2(motor2Pins[0], motor2Pins[1], motor2Pins[2], potenciometro[1], 1);

void configurarPines() {

```

```

// Inicializar motores
motor1.iniciar();
motor2.iniciar();

// Configurar pines de LEDs
for (int pin : ledPins) {
    pinMode(pin, OUTPUT);
}

// Configurar pines de potenciómetros
for (int pin : potenciometro) {
    pinMode(pin, INPUT);
}

// Configurar pines de DIP Switch
for (int pin : dipSwitchPins) {
    pinMode(pin, INPUT_PULLDOWN);
}
}

void controlarLeds(bool encender) {
    for (int pin : ledPins) {
        //aquí aplicamos la operación NOT, para leds con compuerta not
        if(!encender){
            digitalWrite(pin, HIGH);
        }else{
            digitalWrite(pin, LOW);
        }
    }
}
}

```

```

void controlarMotor(MotorContinua& motor, int enableDip, int dirDip) {
    if (digitalRead(enableDip) == HIGH) {
        Serial.println("Iniciando motor");
        bool direccion = digitalRead(dirDip) == HIGH;
        motor.cambiarDireccion(direccion);
        if (direccion==true){
            Serial.println("Direccion 1");
        }else{
            Serial.println("Direccion 2");
        }
        motor.encender(); //se reenciende motor
        motor.setVelocidad(); //actualiza l avelocidad de nuevo
        Serial.print("Velocidad motor: ");
        Serial.println(motor.getVelocidad());
    } else {
        motor.apagar();
    }
}

void setup() {
    Serial.begin(115200);
    configurarPines();
    Serial.println("Iniciando control de motores...");
}

bool estadoLeds=false;

void loop() {

for (int i = 0; i < 8; i++) {
    Serial.print("DIP ");
    Serial.print(i);

```

```

        Serial.print(": ");
        Serial.println(digitalRead(dipSwitchPins[i]));
    }
    delay(100);

    if (digitalRead(dipSwitchPins[7]) == LOW) {
        motor1.apagar();
        motor2.apagar();
        controlarLeds(false);
        Serial.println("PARO DE EMERGENCIA ACTIVADO");
        estadoLeds=false;
        controlarLeds(estadoLeds);
        while (digitalRead(dipSwitchPins[7]) == LOW) {
            delay(100); // Espera hasta que se libere
        }
        return;
    }

    // Controlar ambos motores
    controlarMotor(motor1, dipSwitchPins[0], dipSwitchPins[1]);
    controlarMotor(motor2, dipSwitchPins[2], dipSwitchPins[3]);

    // Control de LEDs
    estadoLeds = (digitalRead(dipSwitchPins[5]) == HIGH);
    controlarLeds(estadoLeds);

    delay(10);
}

```

Comentarios sobre el desarrollo.

El desarrollo es incremental, y se han realizado diversas actividades con éxito, se realizó la documentación con éxito debido a los tiempos y a errores durante la creación del código , así como la limitación en la simulación , falta de modulo puente H , ya que la plataforma aún no tiene módulos que eran necesarios por lo que se señalaron usando los leds como indicadores , estos también pasaron al circuito en físico para indicar lo que ocurría entre el microcontrolador y el puente H , se redujo la capacidad de control del sistema de microcontrolador esp-32 a solo control de motores de corriente continua , con el ADC, al ser un esp32-s3 cuenta con bastantes puertos disponibles de ADC y PWM ,para poder controlar bastantes más motores el objeto que se creó para el control es perfectamente adaptable a otras plataformas(microcontroladores) compatibles con el frameworks Arduino, bajo algunas adaptaciones dependiendo de los métodos para usar adc y pwm .

CAPITULO 4. COMPETENCIAS DESARROLLADAS

Competencias técnicas adquiridas.

Durante el desarrollo del proyecto, se adquirieron habilidades específicas relacionadas con:

Uso de hardware: Configuración y programación del ESP32-S3, incluyendo el manejo de entradas/salidas digitales y analógicas mediante las herramientas Arduino Framework y PlatformIO.

Herramientas de simulación: Uso intensivo de Wokwi como entorno para simular prototipos antes de implementar en hardware físico.

Programación de periféricos:

Lectura de pines analógicos utilizando ADC.

Generación de señales PWM para control de dispositivos.

Gestión de proyectos ágiles: Aplicación del marco SCRUM para la planificación y entrega incremental del proyecto.

Competencias metodológicas

Análisis y diseño: Identificación de necesidades funcionales y su implementación en el diseño del prototipo en línea.

Adaptación ágil: Capacidad para aprender herramientas y metodologías sobre la marcha, como el aprendizaje rápido de nuevas plataformas y lenguajes de programación relacionados con el ESP32-S3.

Optimización del tiempo: Ajuste de tiempos estimados respecto a los tiempos reales para mejorar la precisión en futuras planificaciones.

Competencias blandas

Trabajo colaborativo: Discusión efectiva para la selección del tema y definición de objetivos.

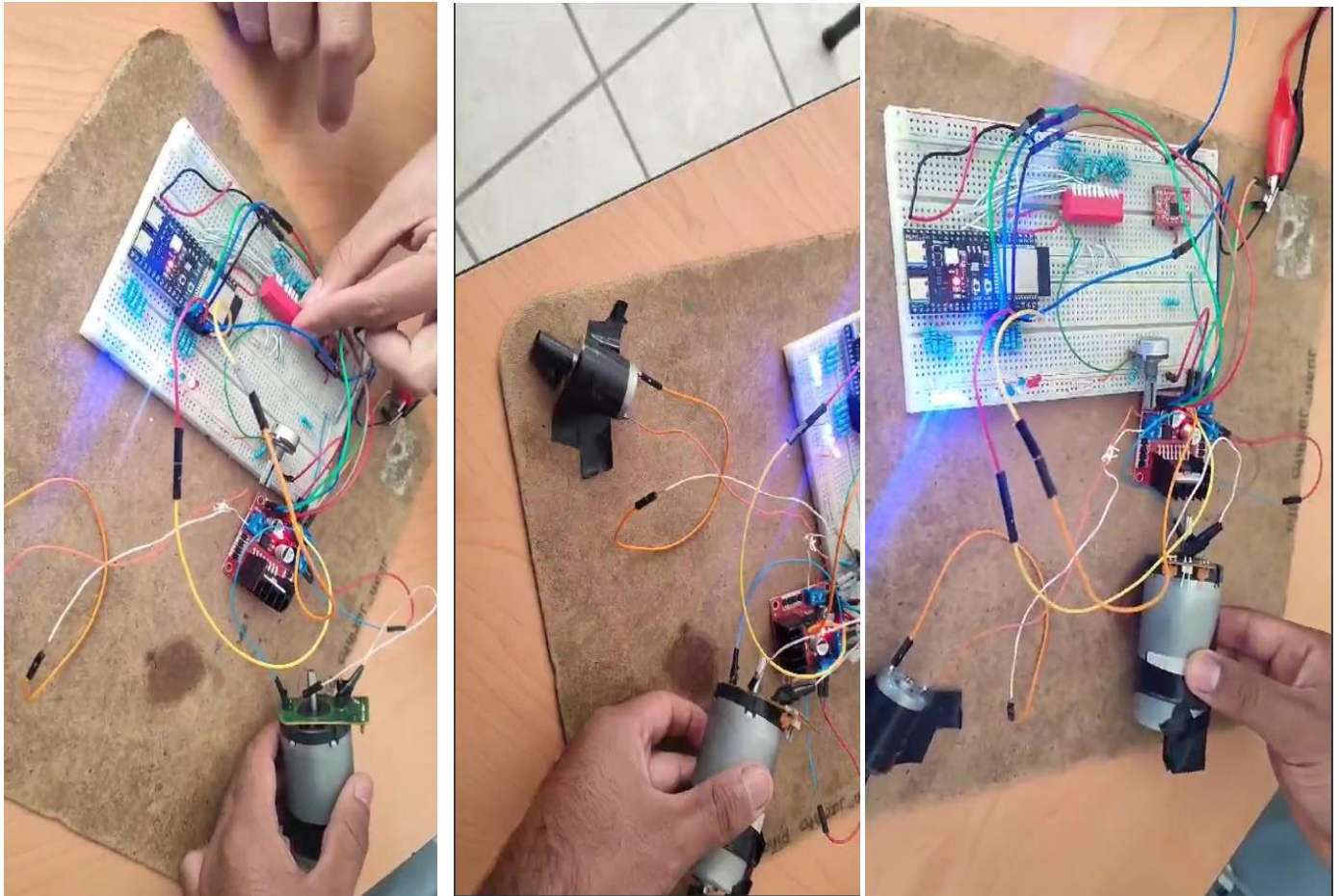
Resolución de problemas: Superación de obstáculos técnicos relacionados con compatibilidades (como la inexistencia del DAC en el ESP32-S3 o diferencias en la funcionalidad de ciertos métodos para el control de los pines analógicos).

Comunicación técnica: Documentación estructurada de cada fase, desde la investigación hasta la implementación.

Comentarios: El desarrollo de este proyecto permitió consolidar habilidades tanto técnicas como metodológicas. Una de las lecciones principales fue entender el enfoque de "aprendizaje sobre la marcha", que es característico del desarrollo ágil. A medida que el proyecto avanzaba, surgieron nuevos desafíos que requerían investigaciones rápidas y decisiones oportunas, lo que mejoró las competencias en resolución de problemas y adaptación a entornos dinámicos.

CAPITULO 5. Resultados y conclusiones

5.1 RESULTADOS.



*Figura 45 Evidencias del proyecto
en funcionamiento*

El circuito funciono con éxito , incluso mejor que en el simulador , puesto que el simulador no contaba con todas las herramientas, es mejor verlo en físico como se controlaba con el microcontrolador los motores usando los 17v , incluso en las pruebas , se pudo , subir el voltaje a 24V.

5.2 CONCLUSIONES DEL PROYECTO.

Martin: Participar en el desarrollo del sistema de control de potencia fue una experiencia reveladora. Me enfoqué en la integración y en asegurar que todo funcionara sin problemas, se lidero el avance del proyecto. Fue gratificante ver cómo el ESP32 se podía manejar motores de alta potencia de manera segura. También supervisé las pruebas y validaciones del prototipo, asegurándome de que se cumplieran todos los requisitos técnicos y operacionales. Fue un reto, pero aprendí muchísimo sobre control de calidad y gestión de proyectos.

Carlos: Trabajar en este proyecto me permitió profundizar en mis conocimientos de programación orientada a objetos y la programación de microcontroladores. Me encargué de programar y de implementar el framework de Arduino en Visual Studio Code, lo cual resultó ser una gran sorpresa, que existiera un proyecto de código abierto y gratis para programar diversos dispositivos en VS Code Ayudé a optimizar el proceso de simulación en línea, lo que facilitó un entorno de desarrollo y prueba mucho más robusto para el equipo. Fue increíble ver cómo nuestras ideas se materializaban en un código eficiente y modular."

Eric: Descubrir y utilizar herramientas de simulación avanzadas fue uno de los aspectos más emocionantes de este proyecto para mí. Uso del diseño en un simulador para el armado del circuito, pudiendo usar una extensión para visualizar el diseño que estaba alojado en online (WOKWI Community es ONLINE) para Visual Studio Code, lo cual fue vital para el éxito de nuestro trabajo, Proteus no tiene ESP32 nativamente, ni tiene librerías, probablemente es por el uso de ".elf" en lugar de ".hex" Me encargué de la creación y configuración del decenio en el simulador, permitiendo al equipo realizar pruebas detalladas y precisas. Aprendí mucho sobre simulación y cómo mejorar nuestros procesos de prueba.

Ruben: El resultado fue exitoso se logró controlar con motores de potencia mas de 20v, usando el esp32 que maneja 3.3 v, es decir se logro un control de potencia, se logro programar usando el entorno de trabajo configuración (framework), librerías y algunos métodos de Arduino en Visual Studio Code, se descubrió un potente simulador en línea el cual también tiene una versión de paga local para visual studio code, se creó un diseño en un simulador, y se uso lo aprendido en la materia de programación usando el paradigma orientado a objetos. Se trato de investigar la programación del ESP-32 con RUST, pero para poder pasarlo es necesaria una especialización más profunda pues el lenguaje es de nivel mas bajo que Arduino, y requiere mas configuraciones sobre el hardware, por eso se descartó y se usó mejor Arduino/C++ cpp.

Bibliografías

- AG, E. (2021). *Agelectronica*. Obtenido de Agelectronica: <http://www.agelectronica.com>
- angelmicelti. (13 de 2 de 2023). *Motor D.C.* Obtenido de Motor D.C.: https://angelmicelti.github.io/4ESO/CYR/motor_dc.html
- Bassi, A. (15 de 05 de 2021). *gotoiot.com*. Obtenido de Instalación de PlatformIO en VS Code: https://www.gotoiot.com/pages/articles/platformio_vscode_installation/index.html
- CODE, V. S. (s.f.). *VISUALSTUDIO*. Obtenido de VISUALSTUDIO: <https://code.visualstudio.com/>
- CORPORATION, H. E. (2021). *HIOKI*. Obtenido de HIOKI: <https://www.hioki.com/us-es/learning/electricity/resistance.html>
- D.R, O. (2024). *OCC* . Obtenido de <https://www.occ.com.mx/empleos/de-idiomos/>
- Electrónica, J. (2017 - 2022). *Julpin*. Obtenido de Julpin: <https://www.julpin.com.co/inicio/resistencias-y-potenciometros/708-potenciometro-de-100k.html#:~:text=El%20potenci%C3%B3metro%20tiene%20una%20resistencia,solo%20o%20con%20alguna%20vibraci%C3%B3n.>
- ESPRESIFF. (2024). *ESPRESIFF.com*. Obtenido de <https://www.espressif.com/>
- ESPRESSIF. (s.f.). <https://docs.espressif.com>. Obtenido de <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/establish-serial-connection.html>
- FÁCIL, P. (2024). *El blog del arduino*. Obtenido de El blog del arduino: <https://programarfácil.com/blog/arduino-blog/el-potenciometro-y-arduino/>
- Forbes, M. (5 de 12 de 2023). *Hablar ingles es cada vez mas importante* . Obtenido de Forbes: <https://forbes.com.mx/hablar-ingles-importante-trabajo-mexico-de-los-peor-calificados-de-latam/>
- Geek, B. (2005-2024). *blog.bricogreek*. Obtenido de blog.bricogreek: <https://blog.bricogeek.com/noticias/programacion/como-programar-arduino-esp8266-y-esp32-con-platformio/>
- Geekfactory. (3 de 6 de 2018). *L298N Modulo Puente H*. Obtenido de L298N Modulo Puente H: <https://www.geekfactory.mx/producto/l298n-modulo-puente-h-doble-control-de-motor/?srsltid=AfmBOoqY5ydAvavnNBsxbWktfHgvIfklFnuYJdaiLhQP9-Yj0tMPvtti>
- HubSpot. (2024). *hubspot*. Obtenido de <https://www.hubspot.es/>
- hutscape.com*. (s.f.). Obtenido de (External Wakeup With Arduino on ESP32-S3): <https://hutscape.com/tutorials/external-wakeup-arduino-esp32s3>
- INVENTABLE.EU. (2010-2020). *Inventable EU. La electrónica simple y clara*. Obtenido de <https://www.inventable.eu/2018/03/13/como-conectar-un-potenciometro-de-volumen/>

- jBo. (s.f.). *DroneBot Workshop Forums*. Obtenido de <https://forum.dronebotworkshop.com/tools-test-equipment/esp32-board-too-fat-for-my-breadboard/>
- KeekNetic. (2002-2024). *Resistencias, what is?* Obtenido de Resistencias, what is?
- Luisllamas. (04 de 12 de 2022). *wokwi simulador*. Obtenido de wokwi simulador: <https://www.luisllamas.es/wowki/>
- Medium. (s.f.). *Medium*. Obtenido de Cómo usar el módulo Puente H L298N con la tarjeta ESP32: <https://medium.com/@SyrkCorp/c%C3%B3mo-usar-el-m%C3%B3dulo-puente-h-l298n-con-la-tarjeta-esp32-f7becce9714>
- Mexico, G. d. (17 de 10 de 2024). *Observatorio digital*. Obtenido de Observatorio digital: https://www.observatoriolaboral.gob.mx/static/preparate-empleo/Articulo_nivel_ingles.html
- Mouser. (31 de 08 de 2021). *ESP-32 Desarrolladores*. Obtenido de ESP-32 Desarrolladores: <https://www.mouser.mx/new/espressif/espressif-esp32-s3-dev-kits/>
- PlatformIO. (2014). *Arduino*. Obtenido de <https://docs.platformio.org/en/latest/frameworks/arduino.html>
- platformio.org. (20 de NOV de 2024). *marketplace.visualstudio.com*. Obtenido de <https://marketplace.visualstudio.com/items?itemName=platformio.platformio-ide>
- ppepeenergy. (2024). *el blog de pepeenergy*. Obtenido de el blog de pepeenergy: <https://www.pepeenergy.com/blog/glosario/definicion-resistencia-electrica/>
- RedHat. (02 de 12 de 2024). *Que es la automatizacion*. Obtenido de Que es la automatizacion: <https://www.redhat.com/es/topics/automation>
- Rossetastone. (17 de 10 de 2024). *rosettastone*. Obtenido de rosettastone: https://es.rosettastone.com/comprar/special-offer-199/?gad_source=1&gclid=CjwKCAjw68K4BhAuEiwAylp3khcPp5H6lUN2lMzLJSvlgRsr9wWbCcDkrlFtZPkxliDxy05lObYnixoC8EEQAvD_BwE
- SDI. (11 de 2 de 2020). *MOTOR DC*. Obtenido de MOTOR DC: <https://sdindustrial.com.mx/blog/que-es-un-motor-dc-y-para-que-sirve/>
- Services, F. M. (2020). *Speak English Now*. Obtenido de Speak English Now: <https://speakenglishnow.club/estudiar-ingles-la-importancia-de-los-habitos/>
- teknei. (02 de 02 de 2024). *Descubre que son los sistemas de automatizacion*. Obtenido de Descubre que son los sistemas de automatizacion: <https://www.teknei.com/mx/2020/08/13/sistemas-de-control-y-automatizacion-industrial/>
- toefl. (2024). *Tomadores en la prueba toefl*. Obtenido de toefl: <https://www.es.ets.org/toefl.html>
- Toluca, I. T. (s.f.).
- Toluca, I. T. (3 de 8 de 2024). *Ingenieria TIC'S*. Obtenido de Ingenieria TIC'S: <https://www.tolucatecnm.mx/programa/10/ingenieria-mecatronica>

TuEquipoSeo. (2014). *Descubre Arduino*. Obtenido de Descubre Arduino: <https://descubrearduino.com/esp32-modulo-esp32-wroom-gpio-pinout/>

uveg. (17 de 10 de 2024). *ingles*. Obtenido de uveg: <https://uveg.edu.mx/index.php/es/ingles>

UVM. (2024). *UVM*. Obtenido de <https://blog.uvm.mx/empleos-que-requieren-un-buen-nivel-de-ingles>

Vasco, U. d. (s.f.). Obtenido de <https://www.ehu.eus/es/home>

vazanza. (23 de 2 de 2021). *Especificaciones ESP32*. Obtenido de Especificaciones ESP32: <https://vasanza.blogspot.com/2021/07/especificaciones-del-modulo-esp32.html>

wokwi. (02 de 11 de 2024). *Wokwi*. Obtenido de Wokwi: <https://wokwi.com/>